Theoretical and Simulated Design of an Autonomous Beach-Cleaning Robot

Yunhu Nam[1] **and Rajit Chatterjea**[2]

Received June 25, 2025
Accepted October 16, 2025
Electronic access December 15, 2025

This paper presents a conceptual design and theoretical control systems modeling framework for an autonomous beach-cleaning robot, engineered to operate in the challenging and dynamic coastal envi- ronment. The proposed robot architecture integrates a solar-powered energy system, a comprehensive suite of sensors (multi-spectral, LiDAR, GPS, IMU, ultrasonic), and multiple actuators (tracks, adaptive suspension, debris collection mechanisms) coordinated by a central microcontroller. A modular state-space model is developed to conceptually capture the dynamics and interactions of all major subsystems, expressed in the canonical form $X = AX + BU, Y = CX + DU$, where the state vector encapsulates battery status, robot kinematics, sensor readings, and actuator states. This modeling approach enables the systematic design of feedback controllers and supports stability analysis based on established Lyapunov theory principles, as detailed from classical and modern control literature. The architecture includes a proposed real-time IoT dashboard for remote supervision and command. By leveraging a rigorous theoretical control systems approach, this work lays a foundation for robust, adaptive, and extensible robotic solutions to environmental cleanup, providing a framework for future research and development toward practical implementation, such as cooperative multi-robot operation and advanced control strategies.

Introduction

Marine plastic pollution has become one of the most pressing environmental challenges of the twenty-first century. A substantial fraction of this plastic waste accumulates on coastlines, where it threatens ecosystems, public health, and local economies dependent on tourism and fisheries. Manual beach cleanups, while effective at small scales, are labor-intensive, episodic, and cannot keep pace with the continuous influx of debris. Large-scale mechanical cleanups, on the other hand, risk disrupting fragile dune ecosystems and are often economically unsustainable.

In this paper we focus specifically on the problem of plastic debris removal on sandy beaches. By narrowing the scope to this class of waste, we address both the environmental urgency of micro- and macro-plastic pollution and the technical challenges that arise from operating in deformable, unpredictable terrains. While our methods and insights may be extendable to general-purpose cleaning in other outdoor or semi-structured environments, our primary motivation is the automated, sustainable removal of plastics such as bottles, packaging, and fishing gear fragments from beaches.

Defining the scope early serves two purposes. First, it clarifies that the proposed robot is not intended for sweeping all types of debris (e.g., organic seaweed, driftwood, or general litter in urban contexts), but rather prioritizes lightweight plastics that are harmful to marine life and often overlooked by conventional cleanup approaches. Second, it emphasizes that the environmental conditions of sandy beaches uneven surfaces, variable compaction, and dynamic obstacles fundamentally shape the robots dynamics and control design. These conditions motivate the nonlinear modeling and adaptive control methods developed later in this paper.

By situating our contribution within the concrete challenge of plastic debris removal, we not only respond to a critical ecological and societal need, but also provide a well-defined domain in which to test and validate advances in nonlinear control, environmental robotics, and sustainability-driven design. The automation of environmental cleanup tasks, such as beach cleaning, presents unique challenges in robotics and control systems engineering. The dynamic, unstructured nature of the beach environment, characterized by variable terrain, unpredictable obstacles, and heterogeneous debris, necessitates a robust, modular, and adaptive control architecture. This work presents the design and modeling of an autonomous beach-cleaning robot, emphasizing a systematic control systems approach.

At the core of the robots architecture is a hierarchical control system integrating multiple sensing, actuation, and computation subsystems. The robot is powered by a Maximum Power Point Tracking (MPPT) solar array with battery storage, providing energy to a suite of sensors (multi-spectral, LiDAR, GPS, IMU, ultrasonic) and actuators (caterpillar tracks, adaptive suspension, rotating sieve, vacuum, magnetic separator). The central

[1] *West Torrance High School, USA*
[b] *AMathematics, The University of Southern California, USA*

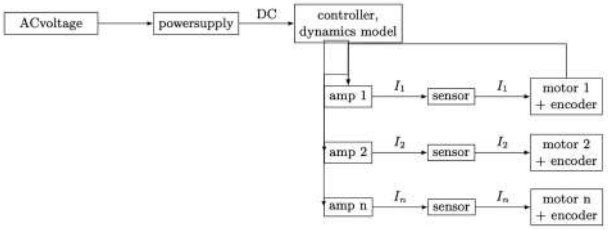


Fig. 1 Compact block diagram of multi-motor current control system.

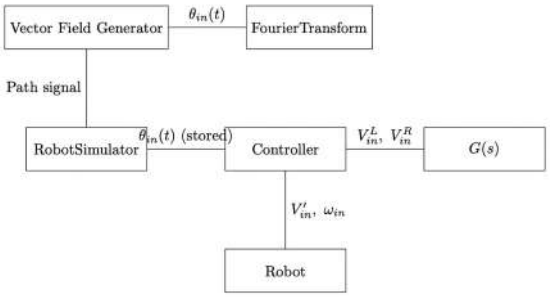


Fig. 2 Compact version of the robot Control system diagram.

microcontroller (STM32/ESP32) acts as the supervisory controller, executing sensor fusion, motion planning, and actuator coordination.

Figure 1 illustrates the robot control system structure showing the current flow. The overall system can be mathematically represented in the standard state-space form:

$$\dot{X}(t) = AX(t) + BU(t) \tag{1}$$

$$Y(t) = CX(t) + DU(t) \tag{2}$$

where $X(t) \in \mathbb{R}^n$ is the state vector encompassing battery state-of-charge, robot kinematics, sensor states, and actuator positions; $U(t) \in \mathbb{R}^m$ is the input vector including remote commands, environmental disturbances, and actuator setpoints; $Y(t)$ is the output vector representing measurable quantities such as robot position, debris detection, and system status.

The A matrix encodes the internal dynamics of each subsystem and their interactions, while B maps control inputs to state changes. For example, the motion subsystem is governed by:

$$\dot{x}_{13} = -dx_{13} + eu_{\text{motor}} \tag{3}$$

where x_{13} is the track velocity and u_{motor} is the motor control input. The robot's position, derived from sensor fusion (LiDAR/GPS/IMU), evolves as:

$$\dot{x}_4 = x_{13}\cos(x_8), \quad \dot{x}_5 = x_{13}\sin(x_8) \tag{4}$$

with x_8 representing orientation.

Hierarchical Control System Architecture

To clarify organization of the hierarchical control system referenced in the introduction, we present Figure 3, which illustrates interaction between the STM32/ESP32 microcontrollers, sensing modules, actuation subsystems, and the IoT dashboard.

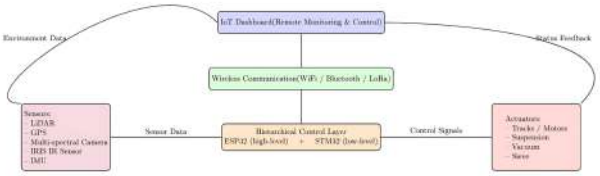


Fig. 3 Hierarchical control System Architecture.

Literature Review

1. In Comprehensive Review of Solar Remote Operated Beach Cleaning Robot by A. Kumar, P. Sharma, and R. Singh, the authors construct a beach-cleaning robot with optimal utilities suited for efficiency and independence, such as solar panels, lithium-ion battery, conceptual description of a camera utilized for navigation, and a conveyor belt system[1]. Our theoretical design aims to achieve the same task as this design, but our conceptual design of a vacuum-filtration system boasts an ideally more efficient path for debris to be collected, due to an increased force by the vacuum. In addition, our theoretical application of LiDAR working alongside a machine learning assisted camera creates a heightened awareness of navigation by concept.

2. In Remote Controlled Road Cleaning Vehicle by R. Patel and S. Desaithe, the authors provide a hardware component list essential for a remote controlled road cleaning vehicle[2]. Specific components listed are a DC power supply (similar to our proposal of a lithium-ion battery usage), microcontroller for essential machine interface and control, and a brush mechanism for collecting trash. Our paper proposes a robot design that proposes a conceptual design of a vacuum-filtration system, which boasts an ideally more efficient path for debris to be collected,

due to an increased force by the vacuum compared to the brush mechanism. In addition, our theoretical application of LiDAR working alongside a machine learning assisted camera creates the possibility for operating and navigating with independence, instead of using remote control.

3. In Autonomous Robotic Street Sweeping: Initial Attempt for Curbside Sweeping by M. Bergerman and S. Singh, the authors provide an analysis of an engineered autonomous street sweeping robot, utilizing an IMU & GPS autonomous navigation along with a fisheye dual-camera setup to target trash according to the image renders[3]. Our design offers similar utilizations, but instead of an IMU & GPS system, we utilize a perimeter rendering LiDAR system along with an AEGIS machine-learning assisted camera to accurately differentiate obstacles and trash by concept. Our setting of automation involves an ocean shore, which holds more environmental obstacles than this experiment's flat, curbside setting, requiring the specific technology in this study to automate these obstacles.

4. In Cleaning Robots in Public Spaces: A Survey and Proposal for Benchmarking Based on Stakeholders Interviews by A. Papadopoulos and L. Marconi, the authors provide an analysis of a combination of interviews of automated robot utilization in public spaces, specifying their performance, capabilities, and ideal requirements for implementation[4]. This connects to our justifiable decision of utilizing our Lidar + SLAM system being a necessity towards self navigation and cognition.

5. In Modular Robot Used as a Beach Cleaner by G. Muscato and M. Prestifilippo, the authors demonstrate simulations of camera image processing, avoidance simulation, and a theoretical claw-like trash collecting mechanism within a similar ocean shore cleaning robot[5]. However, our proposal of a theoretical vacuum filtration system considers the possible increase in efficiency of waste collection compared to this paper's claw design.

6. In OATCR: Outdoor Autonomous Trash-Collecting Robot Design Using YOLOv4-Tiny by H. Chen, L and Zhang, J, the authors propose an outdoor autonomous trash-cleaning robot design, with details of control system structure, robot structure, visual waste detection analysis with mathematical interpretations along with simulations using TACO (Trash annotations in context) assist[6]. This paper displays an application using a machine learning database to identify trash in a robot's surroundings, which is a similar approach to our utilization of AEGIS camera waste identification.

7. In Waste Management by a Robot: A Smart and Autonomous Technique by R. Gupta and N. Sharma, the author provides a control system structure for a remote controlled road cleaning vehicle, and even a prototype physical design of the robot[7]. Specific components listed are microcontrollers (arduino) for essential machine interface and control, and a combination of motor drivers and servo motors for a robotic arm for dynamic trash collection. Our robot design implements a variety of innovations expanding on this foundation. The primary focus of unique innovation is the design of a vacuum-filtration system, which boasts an ideally more efficient path for debris to be collected, due to an increased force by the vacuum. In addition, our theoretical application of LiDAR working alongside a machine learning assisted camera creates the possibility for operating and navigating with independence, instead of using remote control.

8. In Beach Sand Rover: Autonomous Wireless Beach Cleaning Rover by S. Iyer and A. Nair, the authors provide a component list, experimental proposal of infrared sensors combined with a GPS system for independent navigation, and physical models for a claw utilizing road cleaning vehicle[8]. Our robot design implements a variety of innovations expanding on this foundation. The primary focus of unique innovation is the design of a vacuum-filtration system, which boasts an ideally more efficient path for debris to be collected, due to an increased force by the vacuum. In addition, our theoretical application of LiDAR working alongside a machine learning assisted camera creates possibility for operating and navigating with a heightened independence for avoiding obstacles than GPS + infrared sensors.

9. In Autonomous Trash Collecting Robot by K. Lee and A. Patel, the authors provide a component list, experimental proposal of infrared sensors combined with ultrasonic sensors for surrounding awareness, and physical models for a suction vacuum trash collecting system[9]. Our robot design implements a variety of innovations expanding on this foundation. The primary focus of unique innovation is the design of a vacuum-filtration system, which boasts an ideally strong and selective accuracy for a variety of debris to be collected, due to the addition of filtration by the vacuum. In addition, our theoretical application of LiDAR working alongside a machine learning assisted camera creates the possibility for operating and navigating with a heightened independence for avoiding obstacles than ultrasonic + infrared sensors.

10. In Autonomous Litter Collecting Robot with Integrated Detection and Sorting Capabilities by A. Rahman and S. Gupta, the authors provide a component list, simulation analysis of infrared sensors combined with ultrasonic sensors for surrounding awareness, along with a camera assisted with TACO (Trash annotations in context) for surrounding trash identification[10]. Our robot design implements a variety of innovations expanding on this foundation. Our theoretical application of LiDAR working alongside a machine learning assisted camera creates possibility for operating and navigating with a heightened independence for avoiding obstacles compared to the ultrasonic & infrared sensor + camera assisted with TACO, which can only identify trash and cannot identify obstacles like humans in an ocean shore context.

11. In Autonomous Detection and Sorting of Litter Using Deep Learning and Soft Robotic Grippers by P. Proenca and P. Simoes, the authors provide models of a dual fin ray finger design for the waste collecting mechanism along with its ar-

chitecture, along with object waste detection analysis utilizing a camera assisted with TACO for surrounding trash identification[11]. Our robot design implements a variety of innovations expanding on this foundation. Firstly, our vacuum filtration design would ensure more force and simultaneous area of intaking, assisted with a filtration of environmental objects in concept. Second, our theoretical application of LiDAR working alongside a machine learning assisted camera creates possibility for operating and navigating with a heightened independence for avoiding obstacles than ultrasonic and infrared sensor + camera assisted with TACO, which can only identify trash and cannot identify obstacles like humans in an ocean shore context.

12. In AI for Green Spaces: Leveraging Autonomous Navigation and Computer Vision for Park Litter Removal by D. Kim and J. Martinez, the authors provide a concise breakdown and analysis of a park litter removal robot[12]. This design consists of a GPS + Slam under a precise algorithm for autonomous navigation. The design also analyzed 3 different TACO datasets (Trash annotations in context) to determine the most efficient computer vision processing for trash collection. In addition, the final design model displayed utilization of various trash collecting devices, with a conclusion of a rake pickup mechanism giving greatest results. Our robot design implements a variety of innovations expanding on this foundation. Firstly, our vacuum filtration design would ensure more force and simultaneous area of intake, assisted with a filtration of environmental objects in concept. This counters the issue of the analysis of vacuum tested in this paper's model, where it was too small leading to the inability of collecting big objects, along with a lack of filtration, which gives the ability to selectively avoid collecting environmental objects. Our vacuum system is simulated to be drastically wider with a stronger force. Second, our theoretical application of LiDAR working alongside a machine learning assisted camera creates possibility for operating and navigating with a heightened independence for avoiding obstacles than ultrasonic and infrared sensor + camera assisted with TACO, which can only identify trash and cannot identify obstacles like humans in an ocean shore context.

13. In BeWastMan IHCPS: An Intelligent Hierarchical Cyber-Physical System for Beach Waste Management by M. Rizzo and G. Testa, the authors present BIOBLU as a cyber-physical framework for waste-collecting robots, integrating vision-based litter detection with ML, hierarchical control across robotedgecloud layers, GNSS/SLAM-based navigation, modular waste-handling mechanisms, sustainable power solutions, and geotagged data analytics to enable efficient, autonomous, and environmentally sustainable cleanup operations[13]. Our robot provides an abstract design of a filtration vacuum system, with ability to filter out environmental objects and exceeding size items from mass collectible plastic debris, with a compelling force by context. Our autonomous navigation components are near identical in terms of function.

14. In Design a Beach Cleaning Robot Based on AI and Node-RED Interface for Debris Sorting and Monitor the Parameters by T. Mallikarathne and R. Fernando, the authors provide a beach cleaning robot model with a debris-sifting mechanism, AI model utilizing an identification system of collected debris and a Lidar + GPS model for autonomous navigation[14]. Our model proposes a similar but an alternate approach to the debris collecting device: a filtration vacuum system, with an ability to filter out environmental objects and exceeding size items from mass collectible plastic debris, with a compelling force by context.

15. In Trash Collection Gadget: A Multi-Purpose Design of Interactive and Smart Trash Collector, by H. Zhou and L. Wang, the authors offer a physical prototype of a robot with a similar task: to portably clean ocean shores of its abundance in plastic debris[15]. This paper also provides experimental data with trash collection rates varying on displacement. This design boasts a portable conveyor belt system and track-equipped wheels. Our robot design provides a more independent system, with theoretical design of applying components regarding autonomous navigation with equipped awareness, along with a filtration vacuum system for efficient, powerful, and selective debris collection.

16. In Autonomous Beach Cleaning Robot Controlled by Arduino and Raspberry Pi by R. Mehta and A. Khan, the authors provide a component list of a person-operated beach cleaning machine, with a waste collection mechanism consisting of wiper motor, mesh, and a roller pipe[16]. Our autonomous beach-cleaning robot design is inspired by the mesh applied collector, as while the inspiring paper does not provide specifically detailed reasons for usage, it has inspired the usage of our nets to separate sufficient debris for collection and small and negligible environmental objects such as sand within our vacuum tube collector.

17. In Design and Fabrication of Beach Cleaning Equipment by J. Patil and V. Sharma, the authors provide a detailed component list and a physical model design of an ocean-shore collection mechanism, equipped with a conveyor chain hook arrangement for scooping[17]. In addition, there has been a 3D CAD model connected along with methodology based on the structure. Our model provides an abstract filtration vacuum mechanism that has the same objective of efficiently collecting waste in its surroundings, but with a greater force and selectivity. While this paper's design has a standard 4-wheel locomotion system, our locomotive system boasts a caterpillar-track platform for stable movement through the sandy ocean shore. Our robot design also approaches a robot capable of autonomy, but utilizing self-navigation through the utilization of a Lidar system along with a machine-learning assisted camera.

18. In Beach Cleaning Robot by K. Suresh and D. Ramesh, the authors display a beach-cleaning robot component list with a control system structure, along with a 3D render CAD model

and a physical prototype for demonstrating the robot structure[18]. The models have a 4-wheel locomotive system, along with a conveyor belt debris collection mechanism. Our robot design expands on both of these innovations by applying a caterpillar track locomotion system to move through the sandy shore in stability. Our design also provides an abstract filtration vacuum mechanism that has the same objective as the conveyor belt system of efficiently collecting waste in its surroundings, but with a greater force and selectivity.

19. In Design and Development of Beach Cleaning Machine by A. Narayan and R. Kulkarnipaper, the authors provide a system design of an autonomous beach cleaning robot consisting of a component list along with a control system structure[19]. The paper also provides hardware design overview details towards certain components, and overviews of the motor control system, garbage recognition algorithm, and the communication interface. Our model provides more detailed component breakdowns and of the autonomous processing of a robot applied on an ocean shore, specifically towards an elevated garbage recognition algorithm under more advanced components.

20. In Smart AI-Based Waste Management in Stations by J. Lee and Y. Chen, the authors provide methodology and experimentation towards applying SLAM and 2-D LiDAR towards autonomous navigation[20]. Specifically, this paper targets Correlative Scan Matching (CSM) towards precise measurements in navigation. In addition, the utilization of a variety of loop closure detection methods towards reducing noise sensitivity is displayed, for an extreme navigation accuracy. Our model provides a stack of GPS and LiDAR measurement models to expand on robot navigation, by specifically employing a fusion network based on Extended Kalman Filter (EKF) within a LiDAR SLAM pipeline. Regarding the topic of autonomous navigation, our paper applies the concept of LiDAR and GPS to an ocean-shore setting for the robot, where human and environmental obstacles are abundant, needing frequent awareness by the robot to safely avoid interaction with them.

21. In Sweeping Robot Based on Laser SLAM by X. Zhang and Y. Liu, the authors provide a SLAM simulation towards robot self navigation, utilizing a Gmapping an algorithm based on the Rao-Blackwellized particle filtering, within the Gazebo simulating software towards precise LiDAR mapping and navigation tests[21]. Our model provides a stack of GPS and LiDAR measurement models to expand on robot navigation, by specifically employing a fusion network based on Extended Kalman Filter (EKF) within a LiDAR SLAM pipeline. Regarding the topic of autonomous navigation, our paper applies the concept of LiDAR and GPS to an ocean-shore setting for the robot, where human and environmental obstacles are abundant, needing frequent awareness by the robot to safely avoid interaction with them.

22. In Design and Experimental Research of an Intelligent Beach Cleaning Robot Based on Visual Recognition by Q. Jiang, X. Wang, and X. Zhang, the authors applied reinforcement learning towards experimenting with Khepera IV's omniscience sensors to amplify the robot's obstacle identifying capabilities, increasing its autonomous function[22]. While this paper uses 8 infrared sensors for its simulation analysis, our model provides a stack of GPS and infrared LiDAR measurement models to expand on robot navigation, by specifically employing a fusion network based on Extended Kalman Filter (EKF) within a LiDAR SLAM pipeline. Regarding the topic of autonomous navigation, our paper also applies this objective to an ocean-shore setting for the robot, where human and environmental obstacles are abundant, needing frequent awareness by the robot to safely avoid interaction with them. Our simulations regarding additional variables, such as a Machine-learning assisted camera for easier obstacle identification, has been considered for our python simulations.

23. In Diseo e Implementacin de un Prototipo de Robot para la Limpieza de Playas by F. Martnez and D. Snchez, the authors propose a sustainable beach-cleaning robot design, specifically towards its utilization of a rake sand-sifting mechanism towards collecting debris[23]. This paper also provides a motion torque analysis. Our robot design implements a vacuum filtration design expanding on this foundation. This mechanism would ensure more force and simultaneous area of intake, assisted with a filtration of environmental objects in concept. Our vacuum system is simulated to be drastically wider with a stronger force.

24. In Design and Implement of Beach Cleaning Robot by M. Rahman, T. Hasan, and S. Akterpaper, the authors propose a beach cleaning robot design with a conveyor belt debris collection mechanism, along with a breakdown of the electrical flow control system architecture satisfying the robot operations[24]. Our robot design implements a vacuum filtration design expanding on this foundation. This mechanism would ensure more force and simultaneous area of intake, assisted with a filtration of environmental objects in concept. Our vacuum system is simulated to be drastically wider with a stronger force.

25. In EcoBot: An Autonomous Beach-Cleaning Robot for Environmental Sustainability Applications by F. M. Talaat, A. Morshedy, M. Khaled, and M. Salem, the authors present the design and implementation of EcoBot, an autonomous beach-cleaning robot utilizing a mechanical arm debris collection system, and YOLO (You Only Look Once) computer vision for easy debris identification in its surroundings, helping navigation[25]. The robot also utilizes a variety of sensors, including ultrasonic sensors for a more assisted, autonomous navigation. This paper provides a control system architecture, waste detection analytics, and hardware design models of the robot. Our robot design implements a variety of innovations expanding on this foundation. Firstly, our vacuum filtration design would ensure more force and simultaneous area of debris intake, assisted with a filtration of environmental objects in concept. Our vacuum system is conceptualized towards higher efficiency com-

pared to the mechanical arm debris collection system. Second, our theoretical application of LiDAR working alongside a machine learning assisted camera creates possibility for operating and navigating with a heightened independence for avoiding obstacles, which is a similar alternate approach compared to EcoBot's YOLO application and ultrasonic sensor system for autonomous navigation.

Detailed Description of the Robot Control System

The system is modular and consists of a sequence of computational and physical blocks that transform the high-level path planning strategy into executable control commands. Each block is explained below.

Vector Field Generator

This module constructs a vector field across the robot's environment by summing attractive and repulsive forces. The attractive force is directed toward the goal, while the repulsive forces are perpendicular to the attractive vector and modulated by a Gaussian function. The resultant vector $\vec{V}_p$ at each position is given by:

$$\vec{V}_p = \vec{V}_T + \sum_{i=1}^{n} \vec{V}_{T90_i}$$

where $\vec{V}_T$ is the target-directed attractive force and $\vec{V}_{T90_i}$ represents the orthogonal repulsive force due to the i-th obstacle. The output of this block is a direction of motion $\theta_{in}(t)$, defined as:

$$\theta_{in}(t) = \angle \vec{V}_p$$

Clarification: Vector Field Generator for Navigation
The Vector Field Generator computes a navigation direction at every position on the map by combining:

- Attractive force towards the goal, guiding the robots motion.
- Repulsive forces from obstacles, discouraging movement close to them.

Let p_{robot} denote the robots current position and p_{goal} the goal.

Attractive Vector
The normalized attractive vector points directly from the robot to the goal:

$$\vec{V}_{\mathrm{T}} = \frac{p_{\mathrm{goal}} - p_{\mathrm{robot}}}{\|p_{\mathrm{goal}} - p_{\mathrm{robot}}\|}$$

This defines the main direction of travel.

Repulsive Vectors
For each obstacle i at position $p_{\mathrm{obs},i}$:

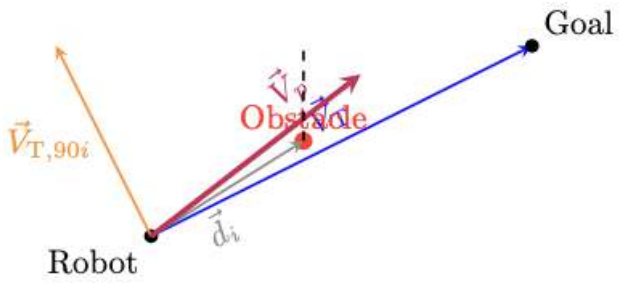
- Compute the vector from robot to obstacle: $\vec{d}_i = p_{\mathrm{obs},i} - p_{\mathrm{robot}}$
- Rotate $\vec{V}_{\mathrm{T}}$ by $+90^\circ$ (counterclockwise) to get a unit vector $\vec{V}_{\mathrm{T},90i}$ perpendicular to the attractive direction.
- The magnitude of the repulsive effect is determined by a Gaussian envelope:
- $w_i = \rho_i \exp\left(-\frac{\|p_{\mathrm{robot}} - p_{\mathrm{obs},i}\|^2}{2\sigma^2}\right)$
- where ρ_i controls repulsion strength, and σ sets the distance scale.
- The total repulsive force is then $w_i \vec{V}_{\mathrm{T},90i}$.

Combined Navigation Vector
The final direction is given by:

$$\vec{V}_p = \vec{V}_{\mathrm{T}} + \sum_i w_i \vec{V}_{\mathrm{T},90i}$$

The robot follows the angle of $\vec{V}_p$, which smoothly guides it towards the goal while veering away from obstacles, but always with a component directed to the goal.

Illustrative Diagram



Caption: At the robot's current location, the blue arrow shows the direction to the goal ($\vec{V}_{\mathrm{T}}$). The orange arrow is the repulsive (perpendicular) component, modulated in strength by Gaussian proximity to the obstacle (red). The purple arrow is the final direction $\vec{V}_p$ which ultimately steers the robot clear of the obstacle while keeping it headed for the goal.

This construction achieves smooth, goal-directed navigation that can safely skirt obstacles without producing the oscillatory "zig-zag" or deadlocks seen in other potential field methods, as repulsive influences only serve to laterally nudge the heading, never to fully reverse it.

Fourier Transform Block

To ensure that the robot is capable of physically executing the desired trajectory, the heading signal $\theta_{in}(t)$ is analyzed in the frequency domain using a Fourier Transform:

$$\Theta_{in}(j\omega) = \mathscr{F}\{\theta_{in}(t)\}$$

This frequency representation is compared against the known bandwidth of the robot's actuators. If the frequency content of the signal exceeds this bandwidth, the robots speed must be reduced to avoid overshooting or oscillation.

Robot Simulator

This block virtually simulates the robots response to the vector field before actual execution. It stores the heading command $\theta_{in}(t)$ and evaluates if the robot can track the path with minimal error. This predictive module prevents infeasible commands from being sent to the physical robot.

Controller

The controller compares the stored heading $\theta_{in}(t)$ with the robots actual orientation to compute angular velocity ω_{in}. Additionally, the forward velocity V'_{in} is calculated using the centripetal acceleration constraint:

$$V'_{in} = \frac{a_{\text{const}}}{\omega_{in}}$$

The computed forward and angular velocities are then transformed into left and right wheel velocities using the transformation:

$$\begin{bmatrix} V_{in}^L \\ V_{in}^R \end{bmatrix} = \begin{bmatrix} 1 & -L/2 \\ 1 & L/2 \end{bmatrix} \begin{bmatrix} V'_{in} \\ \omega_{in} \end{bmatrix}$$

where L is the wheelbase of the robot.

Robot

The robot executes the commands (V'_{in}, ω_{in}) by driving its left and right wheels at the corresponding velocities (V_{in}^L, V_{in}^R). The robot's trajectory is continuously corrected based on the heading and velocity feedback from the controller.

Robot Plant Model $G(s)$

This block models the dynamic behavior of the robot's velocity response. In simulation, a first-order linear system with time delay is used:

$$G(s) = \frac{K_V e^{-T_D s}}{1 + T_1 s}$$

where K_V is a gain factor, T_1 is the system time constant, and T_D is the communication/processing delay. The model is calibrated using step-response experiments and used in simulation to ensure accurate trajectory prediction.

The integration of these blocks ensures the robot navigates complex environments safely and efficiently, while dynamically adapting to its physical constraints.

The modular state-space model enables the systematic design of feedback controllers for each subsystem, as well as the analysis of system stability, controllability, and observability. The IoT dashboard provides a human-in-the-loop supervisory layer, allowing for remote monitoring and command injection.

This control-centric modeling framework supports the implementation of advanced control strategies (e.g., optimal, adaptive, or robust control) and facilitates future extensions, such as multi-robot coordination or learning-based adaptation, within a principled systems engineering context.

Robot Perception-Decision-Action Workflow

A robust perception-decision-action pipeline is central to the autonomous operation of the beach-cleaning robot. The full control architecture, from environmental sensing to mechanical actuation, can be summarized as follows:

Workflow description:

- Sensors: The robot continuously acquires data on terrain, obstacles, debris, and its internal state using its sensor suite.
- Sensor Fusion and Preprocessing: Raw sensor data are fused and filtered to produce robust, high-confidence environment state estimates (e.g., position, nearby obstacles, detected debris).
- Controller/Decision Logic: This module receives the estimated state, plans a motion/path, and generates actuator commands based on integrated models (e.g., vector-field navigation, feedback control, sorting logic).
- Actuators: Commands are transmitted to all robot effectors (tracks, suspension, vacuum/sieve, separator), interacting with the environment.
- Feedback Loop: The results of the actions update the sensory input, closing the loop for continuous, adaptive operation.

This pipeline ensures that the robot autonomously closes the loop between perception, control, and actuation in a robust and extensible fashion. The workflow has been illustrated in Figure 4.

Integration of Multi-Spectral Camera with AEGIS Machine Learning in Control System

The beach-cleaning robot utilizes an advanced multi-spectral camera system equipped with the AEGIS machine learning (ML) application to achieve precise debris identification. This integration plays a critical role in the robot's perception and control architecture.

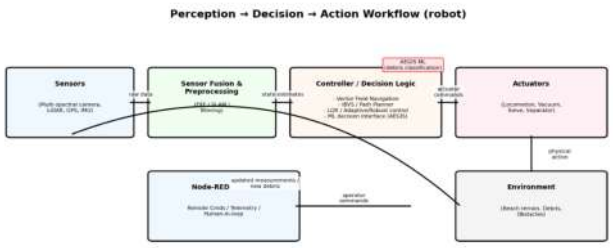


Fig. 4 Workflow Diagram.

State-Space Representation

In the modular state-space modeling framework, the multi-spectral camera sensor output, enhanced via AEGIS ML-based debris classification, is represented as a key sensor state variable denoted by x_3. This state captures the processed debris detection signal within the robot's environment.

Mathematically, the sensor dynamics can be modeled by a first-order linear system with external environmental debris input u_3:

$$\dot{x}_3 = -fx_3 + gu_3,$$

where f represents the sensor decay or filtering rate, and g models the sensor response gain.

5.2 Influence on Robot Commands

The debris detection state x_3 directly impacts actuator commands responsible for waste collection. Specifically, the uptake subsystems for vacuum and sieve adjust their actuation based on the debris detection signal, as reflected by the dynamics:

$$\dot{x}_{15} = -hx_{15} + j\,(u_4 + x_3)\,,$$

$$\dot{x}_{16} = -kx_{16} + lu_4,$$

where x_{15} and x_{16} represent the sieve speed and vacuum pressure states, respectively, and u_4 is the actuator setpoint command.

Here, the inclusion of x_3 in the input to the sieve speed dynamics indicates that increased debris detection results in intensified sieve operation. Thus, the ML-driven sensor state dynamically modulates waste collection mechanisms.

Control Architecture Implications

While the debris detection signal x_3 is part of the overall state vector X, the ML component also interfaces with higher-level decision modules within the STM32/ESP32 microcontroller. The system leverages AEGIS's classification outputs to:

- Differentiate between environmentally harmful debris and benign objects,
- Selectively activate actuators for targeted waste collection,
- Prioritize navigation paths toward debris-congested areas,
- Reduce false-positive activations through adaptive machine learning inference.

This perception-driven feedback creates an adaptive control loop where sensor-derived knowledge informs and adjusts robot actions in real time.

The integration of the multi-spectral camera and AEGIS ML technology is formally encapsulated as a measurable state x_3 in the robot's state-space model. Its influence cascades through the control hierarchy, enhancing vacuum and sieve actuation and enabling perception-informed autonomous behavior.

This design approach ensures robust, selective, and efficient debris cleanup by coupling advanced sensing and machine learning tightly with control system dynamics.

Construction of the Global State Space Model

The state-space representation introduced earlier has the compact form

$$\dot{X}(t) = AX(t) + BU(t),$$

but the process by which subsystem dynamics are combined into the global matrices A and B was not previously detailed. We provide that explanation here.

Subsystem Equations

Each physical subsystem—battery, locomotion, suspension, vacuum/sieve, sensors—is first described by a local state equation of the form

$$\dot{x}_i(t) = A_i x_i(t) + B_i u_i(t),$$

where $x_i \in \mathbb{R}^{n_i}$ are subsystem states, $u_i \in \mathbb{R}^{m_i}$ are local inputs, and $A_i \in \mathbb{R}^{n_i \times n_i}$, $B_i \in \mathbb{R}^{n_i \times m_i}$ are subsystem dynamics matrices. For example:

- Battery model: (x_1, x_2) = [state-of-charge, bus voltage],
- Motion model: (x_4, x_5, x_8, x_{13}) = [position, heading, track velocity],
- Suspension: x_{14},
- Vacuum/sieve: (x_{15}, x_{16}),
- Environmental load: x_3 (disturbance), etc.

Stacking Procedure

The global state vector X is formed by concatenation:

$$X(t) = \begin{bmatrix} x_1^\top & x_2^\top & \cdots & x_k^\top \end{bmatrix}^\top,$$

with dimension $n = \sum_i n_i$. Similarly, the global input vector is

$$U(t) = \begin{bmatrix} u_1^\top & u_2^\top & \cdots & u_k^\top \end{bmatrix}^\top,$$

with dimension $m = \sum_i m_i$.

If subsystems are completely decoupled, the global matrices take block-diagonal form:

$$A = \begin{bmatrix} A_1 & 0 & \cdots & 0 \\ 0 & A_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & A_k \end{bmatrix}, \qquad B = \begin{bmatrix} B_1 & 0 & \cdots & 0 \\ 0 & B_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & B_k \end{bmatrix}.$$

In practice, beach operation introduces couplings between subsystems (e.g., suspension deformation affects locomotion drag; vacuum load affects battery dynamics). These appear as off-diagonal blocks in A and B. For instance, the coupling between track velocity x_{13} and suspension x_{14} is encoded as a nonzero off-diagonal entry $A_{13,14}$.

Visual Schematic

The stacking can be visualized as shown in Figure 5, where local subsystem dynamics are assembled into the global model: In short, the complete state-space model is obtained by stacking all subsystem equations in block form and explicitly adding off-diagonal terms for physical couplings. This method makes the structure of A and B transparent, and provides a modular pathway to update the global dynamics whenever subsystems are refined or re-identified.

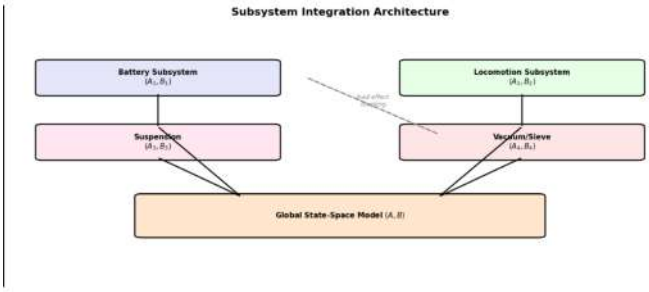


Fig. 5 Illustration of the stacking process: each subsystem contributes its own (A,B) block, and coupling terms generate off-diagonal entries in the global A, B matrices.

Consistent State-Space Model Definitions and Simplified Analysis

The core dynamics of the beach-cleaning robot are captured within a state-space modeling framework. To avoid ambiguity, we provide explicit definitions for each symbol and simplify the presentation to reflect the actual implementation.

State-Space Model Structure

The system is modeled (possibly with both linear and nonlinear terms) as

$$\dot{x}(t) = Ax(t) + Bu(t) + f(x,t) \tag{5}$$
$$y(t) = Cx(t) + Du(t) \tag{6}$$

Where:

- $x(t) \in \mathbb{R}^n$ is the **state vector** (with all main subsystem states)
- $u(t) \in \mathbb{R}^m$ is the **input vector** (external controls, setpoints)
- $y(t) \in \mathbb{R}^p$ is the **output vector** (sensor or monitored outputs)
- A, B, C, D are system matrices of appropriate dimension
- $f(x,t)$ represents potentially nonlinear coupling (e.g., position kinematics)

Explicit State and Input Definitions

For the implemented and simulated model, the principal states are:

$$x(t) = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_8 \\ x_{13} \\ x_{15} \\ x_{16} \end{bmatrix} = \begin{bmatrix} \text{Battery SoC} \\ \text{Bus voltage (V)} \\ \text{Debris detection signal} \\ \text{Position } x \text{ (m)} \\ \text{Position } y \text{ (m)} \\ \text{Orientation } \theta \text{ (rad)} \\ \text{Track velocity (m/s)} \\ \text{Sieve rotational speed} \\ \text{Vacuum subsystem pressure} \end{bmatrix}$$

The input vector is

$$u(t) = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = \begin{bmatrix} \text{Remote track command} \\ \text{Solar charging input} \\ \text{Environmental disturbance (debris)} \\ \text{Actuator setpoint (vacuum/sieve)} \end{bmatrix}$$

Subsystem Dynamics

The system is block modular. The major subsystems are:

Power Subsystem:

$$\dot{x}_1 = -\frac{1}{C_{\text{batt}}} i_{\text{load}} + \frac{1}{C_{\text{batt}}} u_2 \tag{7}$$

$$\dot{x}_2 = -\alpha x_2 + \beta u_2 - \gamma i_{\text{load}} \tag{8}$$

with $x_1 = \text{SoC}$, $x_2 = \text{bus voltage}$, i_{load} a function of actuators.

Debris Sensor Filter:

$$\dot{x}_3 = -f x_3 + u_3 \tag{9}$$

with u_3 the debris/environment disturbance input.

Tracks (locomotion):

$$\dot{x}_{13} = -d x_{13} + e\, u_{\text{motor}} \tag{10}$$

where u_{motor} is a (potentially feedback-controlled) motor command.

Position/heading (kinematics):

$$\dot{x}_4 = x_{13} \cos(x_8) \tag{11}$$

$$\dot{x}_5 = x_{13} \sin(x_8) \tag{12}$$

$$\dot{x}_8 = (\text{steering command}) \tag{13}$$

Vacuum and sieve:

$$\dot{x}_{15} = -h x_{15} + j(u_4 + x_3) \tag{14}$$

$$\dot{x}_{16} = -k x_{16} + l u_4 \tag{15}$$

Summary Table of Variables

Symbol	Meaning	Python Variable
x_1	Battery SoC (state-of-charge)	x1
x_2	Bus voltage	x2
x_3	Debris sensor signal	x3
x_4	x-position	x4
x_5	y-position	x5
x_8	Orientation (heading, radians)	x8
x_{13}	Track velocity	x13
x_{15}	Sieve speed	x15
x_{16}	Vacuum pressure	x16
u_1	Remote (track) command	u1
u_2	Solar charger input	u2
u_3	Debris/environment disturbance	u3
u_4	Vacuum/sieve actuator setpoint	u4

This notation therefore covers all equations used in the theoretical section and in the practical simulation framework, ensuring clarity, reproducibility, and ease of extension for further development.

Simulation Implementation and Role in Design

A comprehensive simulation framework was developed and implemented for the autonomous beach-cleaning robot to verify theoretical models, validate subsystem interactions, and inform overall design choices.

Implementation and Code Structure

The simulator, written in Python, includes explicit code for each major subsystem and their interconnections. Key components and modeling approaches are:

- Power Subsystem: Modeled with differential equations for battery state-of-charge (x_1) and bus voltage (x_2):

$$\dot{x}_1 = -\frac{1}{C} i_{\text{load}} + \frac{1}{C} i_{\text{solar}}$$
$$\dot{x}_2 = -\alpha x_2 + \beta i_{\text{solar}} - \gamma i_{\text{load}}$$

- Motion Subsystem: The velocity of the robots tracks, x_{13}, is governed by

$$\dot{x}_{13} = -d x_{13} + e\, u_{\text{motor}}$$

- Debris Detection Subsystem:

$$\dot{x}_3 = -f x_3 + g\, u_3$$

- Position Kinematics and Navigation:

$$\dot{x}_4 = x_{13} \cos(x_8)$$
$$\dot{x}_5 = x_{13} \sin(x_8)$$
$$\dot{x}_8 = \omega$$

- Vacuum and Sieve Subsystems:

$$\dot{x}_{15} = -h x_{15} + j(u_4 + x_3)$$
$$\dot{x}_{16} = -k x_{16} + l u_4$$

- Integrated System: All equations are stacked forming the system dynamics

$$\dot{X}(t) = f(X(t), U(t))$$

where X aggregates all subsystem states and U is the input vector.

The code utilizes Numpy, scipy.integrate.solve_ivp for integration, and matplotlib for plotting simulation results.

Use of Simulation Outputs in Robot Design

- Subsystem Validation: Each modeled subsystem behaved as predicted, confirming stability (e.g., under Lyapunov analysis) and controllability.
- Integrated Behavior: Simulations produced trajectories for robot position, SoC (State of Charge), and actuator/sensor states, helping validate that control strategies and couplings work as intended.
- Design Decisions Guided by Simulation:
  - Verified that hardware and control constraints (such as actuator bandwidth) were respected through frequency-domain analysis (e.g., Fourier transforms).
  - Confirmed that system outputs, such as tracking performance and robustness, met specification through RMS error analysis and Monte Carlo robustness tests.
  - Supported selection and tuning of controller parameters for safe, efficient, and reliable operation under expected environmental disturbances.
- Reported Results: Figures throughout the paper are direct outputs from the simulation, demonstrating transient responses, SoC and voltage stability, trajectory tracking, and subsystem interplay.

Frequency-Based Speed Adjustment Algorithm

To ensure accurate trajectory tracking without exceeding actuator physical limitations, the robots heading command $\theta_{\text{in}}(t)$ is processed through a frequency-domain analysis. This approach guarantees the actuator bandwidth constraints are respected, preventing overshoot and oscillatory behavior.

Frequency Content Analysis

The desired heading signal $\theta_{\text{in}}(t)$ is converted to the frequency domain via the Fourier Transform:

$$\Theta_{\text{in}}(j\omega) = \mathscr{F}\{\theta_{\text{in}}(t)\}.$$

The magnitude spectrum $|\Theta_{\text{in}}(j\omega)|$ is inspected to identify the bandwidth characteristics of the command.

Actuator Bandwidth Constraint

Let ω_c denote the known cutoff frequency of the robots actuators, determined from actuator specifications and empirical testing. The actuator is assumed capable of faithfully tracking input signals with frequency components below ω_c, while higher frequencies induce errors or mechanical strain.

Speed Reduction Algorithm

The algorithm to adjust the robots speed based on the heading signals frequency content is as follows:

1. Compute the power spectral density (PSD) of $\theta_{\text{in}}(t)$ from $\Theta_{\text{in}}(j\omega)$.
2. Define the high-frequency energy as
$$E_{\text{high}} = \int_{\omega_c}^{\infty} |\Theta_{\text{in}}(j\omega)|^2 \, d\omega.$$
3. Define the total energy as
$$E_{\text{total}} = \int_{0}^{\infty} |\Theta_{\text{in}}(j\omega)|^2 \, d\omega.$$
4. Calculate the ratio
$$r = \frac{E_{\text{high}}}{E_{\text{total}}}.$$
5. If r exceeds a threshold ε (e.g., 5%), indicating significant high-frequency content, reduce the forward speed V'_{in} proportionally according to:
$$V'_{\text{in,new}} = V'_{\text{in,orig}} \times (1 - \kappa r),$$
where κ is a tuning parameter ($0 < \kappa \leq 1$) controlling the sensitivity of speed reduction.
6. Recalculate the heading signal $\theta_{\text{in}}(t)$ corresponding to the new reduced speed and repeat the frequency analysis until $r \leq \varepsilon$.

This iterative speed scaling ensures that the command trajectory remains within actuator capabilities by smoothing out rapid heading changes that exceed physical limits.

Rationale and Literature Context

The approach is consistent with established control strategies where:

- High-frequency reference components cause actuator saturation or tracking errors, compromising system stability.
- Smoothing the input signal by reducing command speeds limits bandwidth demand to actuator feasible range.
- The thresholding parameter ε and reduction factor κ can be tuned based on empirical actuator response and robustness margins.

In effect, this frequency-domain speed adjustment functions as a bandwidth-aware trajectory scaling method, foundational to the robots control framework. By rigorously linking heading signal spectral content to speed commands, the system prevents actuator saturation and promotes robust, precise beach-cleaning operations.

Control Systems Structure and Background

Overall State Space Model for Beach Cleaning Robot

Identifying Subsystems and States

From the overall system diagram, the major subsystems and their representative state variables can be listed as follows:

Subsystem	Example State Variables
MPPT Solar + Battery	Battery SoC (x_1), Bus voltage (x_2)
Multi-Spectral Sensor (AEGIS)	Last detected debris type (x_3)
LiDAR SLAM (Luminar IRIS)	Robot position/heading (x_4, x_5)
GPS	Robot position (x_6, x_7)
IMU	Orientation, angular rates (x_8, x_9, x_{10})
Ultrasonic Sensors	Obstacle distances (x_{11})
STM32/ESP32 Microcontroller	Internal controller states ($x_{12}, \dots$)
Caterpillar Tracks + DC Motors	Track velocities (x_{13})
Adaptive Suspension	Suspension positions (x_{14})
Rotating Cylindrical Sieve	Sieve angle/speed (x_{15})
Universal Vacuum + Mini Vacuums	Vacuum pressure (x_{16})
Magnetic Separator	Separator state (x_{17})
Node-RED IoT Dashboard	Last command received (x_{18})

Input and Output Vectors

Input vector (U) may include:

$$U = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ \dots \end{bmatrix}$$

where, for example:

- u_1: Remote commands (from dashboard)
- u_2: Power input (solar charging)
- u_3: Environmental disturbances (e.g., debris, obstacles)
- u_4: Setpoints for actuators (e.g., motor, vacuum, sieve)

Output vector (Y) may include:

- Robot position, orientation, velocity
- Sensor readings (debris type, obstacles)
- Battery SoC, voltage
- Data sent to dashboard

Example State-Space Equations

Let the overall state vector be:

$$X = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \\ x_9 \\ x_{10} \\ x_{11} \\ x_{12} \\ x_{13} \\ x_{14} \\ x_{15} \\ x_{16} \\ x_{17} \\ x_{18} \end{bmatrix}$$

and the input vector:

$$U = \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \\ \dots \end{bmatrix}$$

The general state-space equations are:

$$\dot{X} = AX + BU$$

$$Y = CX + DU$$

Block Structure of Matrices

Given the modular nature, the A and B matrices are block matrices:

- **A**: Diagonal blocks model internal dynamics of each subsystem (e.g., battery, motors, sensors).
- **Off-diagonal blocks**: Model coupling between subsystems (e.g., how vacuum pressure affects debris collection).
- **B**: Maps control inputs to the states they affect (e.g., motor voltage to track velocity).
- **C**: Maps states to outputs (e.g., which states are sent to the dashboard).
- **D**: Usually sparse; nonzero only if there is a direct, instantaneous effect of input on output.

Example: Expanded Equations for Key Subsystems
a) Power subsystem

$$\dot{x}_1 = -\frac{1}{C}(i_{load}) + \frac{1}{C}(i_{solar})$$
$$\dot{x}_2 = -ax_2 + bi_{solar} - ci_{load}$$

b) Motion subsystem (tracks)

$$\dot{x}_{13} = -dx_{13} + eu_{motor}$$

c) Debris detection (sensor)

$$\dot{x}_3 = -fx_3 + g(\text{debris input})$$

d) Sieve/vacuum

$$\dot{x}_{15} = -hx_{15} + ju_{sieve}$$
$$\dot{x}_{16} = -kx_{16} + lu_{vacuum}$$

e) Robot position (from LiDAR/GPS/IMU fusion)

$$\dot{x}_4 = x_{13}\cos(x_8)$$
$$\dot{x}_5 = x_{13}\sin(x_8)$$

All these equations can be stacked into the large A and B matrices.

Full State-Space Model (Symbolic Form)

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{x}_3 \\ \dot{x}_4 \\ \vdots \\ \dot{x}_{17} \\ \dot{x}_{18} \end{bmatrix} = A \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ \vdots \\ x_{17} \\ x_{18} \end{bmatrix} + B \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ \vdots \end{bmatrix}$$
$$Y = CX + DU$$

where A, B, C, D are constructed by combining the dynamics and interactions of all subsystems.

The following gives the control system structure of the entire robot. 10.2 Nonlinear State-Space Representation

The assumption of a linear time-invariant (LTI) system in modeling the robot dynamics is not fully justifiable for beach environments. Sand deformation, actuator slip, suspension compliance, and debris interactions introduce pronounced nonlinearities that cannot be ignored without compromising model validity. To address this criticism, we reformulate the dynamics using a nonlinear state-space model:

$$\dot{X}(t) = f(X(t),U(t),t), \quad Y(t) = h(X(t),U(t),t), \tag{16}$$

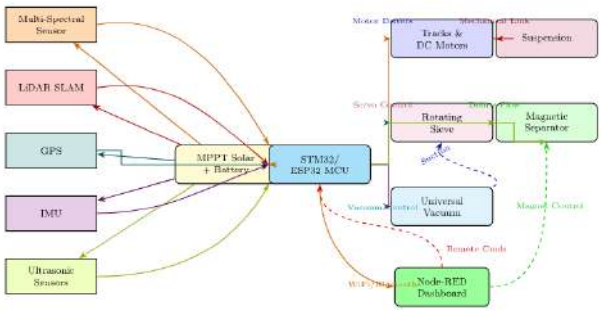


Fig. 6 Overall Control System Structure

where $X(t) \in \mathbb{R}^n$ is the full robot state vector, $U(t) \in \mathbb{R}^m$ is the input vector, and f, h are nonlinear vector fields capturing the coupling between subsystems and the environment.

Example Nonlinear Dynamics For the motion subsystem with track velocity x_{13} and orientation x_8, we include nonlinear slip and drag effects:

$$\dot{x}_{13} = -d\,x_{13} + e\,u_{\text{motor}} - \alpha x_{13}^2 \operatorname{sgn}(x_{13}), \tag{17}$$
$$\dot{x}_4 = x_{13}\cos(x_8), \tag{18}$$
$$\dot{x}_5 = x_{13}\sin(x_8), \tag{19}$$

where $\alpha > 0$ models quadratic velocity drag due to granular resistance in sand. Unlike the linear model, this representation reflects how resistance grows nonlinearly with speed.

Suspension–Track Coupling Suspension deformation (x_{14}) interacts with track dynamics:

$$\dot{x}_{14} = f_1(x_{14}) + g_1(x_{14})u_{\text{susp}}, \tag{20}$$
$$\dot{x}_{13} = f_2(x_{13},x_{14}) + g_2(x_{13},x_{14})u_{\text{motor}}, \tag{21}$$

where f_i, g_i are nonlinear functions obtained from physical modeling or simulational identification. This form allows the model to capture wheel sinkage and terrain-dependent resistance.

Energy Dynamics The battery dynamics incorporate time-varying solar efficiency $\eta_{\text{solar}}(t)$ and nonlinear load dependence:

$$\dot{x}_1 = -\frac{1}{C} i_{\text{load}}(X) + \frac{1}{C}\eta_{\text{solar}}(t) i_{\text{solar}}, \tag{22}$$
$$\dot{x}_2 = -\alpha x_2 + \beta\,\eta_{\text{solar}}(t) i_{\text{solar}} - \gamma i_{\text{load}}(X), \tag{23}$$

where $i_{\text{load}}(X)$ depends nonlinearly on actuator states (x_{13}, x_{15}, x_{16}).

Nonlinear Coupling Stacking all subsystems, we obtain the nonlinear system

$$\dot{X}(t) = AX(t) + BU(t) + f_{\text{nl}}(X(t),U(t),t), \tag{24}$$

where f_{nl} encodes all nonlinear and time-varying terms. This structure preserves the tractability of the LTI backbone while

incorporating essential nonlinearities from sand interaction, suspension deformation, and actuator coupling.

Stability Considerations To assess stability of the nonlinear dynamics, we adopt Lyapunovs direct method. For instance, the quadratic candidate

$$V(X) = \tfrac{1}{2} X^\top P X, \quad P \succ 0 \tag{25}$$

yields

$$\dot{V}(X) = X^\top \big(A^\top P + PA\big) X + X^\top P f_{\mathrm{nl}}(X,U,t). \tag{26}$$

Asymptotic stability is guaranteed if $\dot{V}(X) < 0$ for all $X \neq 0$, which motivates robust or adaptive control designs that explicitly account for nonlinear terms.

Lyapunov–Based Nonlinear Control for Beach-Operating Robot

We refine the dynamics to a control-affine nonlinear form with environment-induced (sand) nonlinearities and matched uncertainties:

$$\dot{X} = AX + Bu + f_{\mathrm{nl}}(X,t), \qquad Y = CX, \tag{27}$$

where $X \in \mathbb{R}^n$, $u \in \mathbb{R}^m$, A, B, C are known from nominal mechanics, and f_{nl} captures terrain-dependent effects (slip, sinkage, quadratic drag, debris) that are not well-approximated by LTI models. We assume the matched structure

$$f_{\mathrm{nl}}(X,t) = B\,\phi(X,t), \tag{28}$$

i.e., the dominant nonlinearities/uncertainties enter through the same channel as the control (a standard assumption for ground robots with actuator-dominated uncertainties).

Reference model and tracking error. Let X_d be a desired (feasible) reference state with dynamics

$$\dot{X}_d = A_r X_d + B_r r(t), \tag{29}$$

where $r(t)$ is a bounded reference command. Define the tracking error $e := X - X_d$. Subtracting (as shown in Equation 29) from (as shown in Equation 27) yields

$$\begin{aligned} \dot{e} &= AX + Bu + B\,\phi(X,t) - \dot{X}_d \\ &= \underbrace{(A - KB)}_{A_c} e + B(u + \phi(X,t)) + \Delta_r(t), \end{aligned} \tag{30}$$

where we add and subtract KBe with a designer-chosen $K \in \mathbb{R}^{m\times n}$ and pack all reference-model mismatch into $\Delta_r(t) := AX_d - \dot{X}_d + KBe$. Choosing $A_c := A - KB$ Hurwitz (e.g., by LQR or pole-placement on (A,B)) stabilizes the nominal linear part.

Linearly parameterized uncertainty and robust residual. We adopt a standard linearly parameterized representation for the leading nonlinearities:

$$\phi(X,t) = Y(X,t)\,\theta^\star + d(X,t), \tag{31}$$

where $Y(X,t) \in \mathbb{R}^{m\times p}$ is a known regressor (e.g., basis of quadratic drag $v|v|$, slip maps, load currents), $\theta^\star \in \mathbb{R}^p$ are unknown constant (or slowly varying) parameters, and $d(X,t)$ collects the residual unmodeled part with known bound $\|d(X,t)\| \le \bar{d}$.

Adaptive–robust control law. Let $\hat{\theta}$ be the parameter estimate and $\tilde{\theta} := \hat{\theta} - \theta^\star$. Consider the control

$$u = -Ke - Y(X,t)\,\hat{\theta} - k_r \,\mathrm{sat}\!\left(\frac{B^\top P e}{\varphi}\right), \tag{32}$$

with gains $k_r > \bar{d}$ and boundary layer $\varphi > 0$, where $P \succ 0$ solves the Lyapunov equation

$$A_c^\top P + PA_c = -Q, \qquad Q \succ 0. \tag{33}$$

The last term in (32) is a continuous robustification (a saturation in the direction $B^\top P e$) that dominates the bounded residual d. The adaptive law is chosen as

$$\dot{\hat{\theta}} = -\Gamma Y(X,t)^\top B^\top P e - \sigma \Gamma \hat{\theta}, \tag{34}$$

with $\Gamma \succ 0$ (adaptation rate) and a small $\sigma > 0$ (σ-modification) to prevent parameter drift under disturbances and noise.

Lyapunov analysis.

Define the composite Lyapunov function

$$V(e,\tilde{\theta}) \;=\; \tfrac{1}{2} e^\top P e \;+\; \tfrac{1}{2} \tilde{\theta}^\top \Gamma^{-1} \tilde{\theta}. \tag{35}$$

Along trajectories of (as shown in Equation 30) with (as shown in Equation 31)-(as shown in Equation 34), and using (as shown in Equation 33),

$$\begin{aligned} \dot{V} &= \tfrac{1}{2} e^\top (A_c^\top P + PA_c) e + e^\top PB\big(-Ke - Y\hat{\theta}\big) \\ &\quad + e^\top PB\Big(-k_r \,\mathrm{sat}\big(\tfrac{B^\top P e}{\varphi}\big) + Y\theta^\star + d\Big) + \tilde{\theta}^\top \Gamma^{-1} \dot{\tilde{\theta}} \\ &= -\tfrac{1}{2} e^\top Q e - e^\top PBKe - e^\top PBY\tilde{\theta} \\ &\quad - k_r\, e^\top PB\,\mathrm{sat}\big(\tfrac{B^\top P e}{\varphi}\big) + e^\top PBd + \tilde{\theta}^\top \Gamma^{-1}(-\dot{\hat{\theta}}) \\ &= -\tfrac{1}{2} e^\top Q e - e^\top PBKe - k_r \,\|\Pi e\|_{1,\varphi} + e^\top PBd \\ &\quad + \tilde{\theta}^\top \Gamma^{-1}\Big(\Gamma Y^\top B^\top P e + \sigma\Gamma\hat{\theta}\Big) \\ &= -\tfrac{1}{2} e^\top Q e - e^\top PBKe - k_r \,\|\Pi e\|_{1,\varphi} + e^\top PBd \\ &\quad + \tilde{\theta}^\top Y^\top B^\top P e + \sigma\,\tilde{\theta}^\top \hat{\theta}. \end{aligned} \tag{36}$$

where $\Pi := \mathrm{diag}(B^\top P)$ projects the error along the input directions and $\|\cdot\|_{1,\varphi}$ denotes the one-norm of the saturated vector. The cross term $-e^\top PBY\tilde{\theta}$ cancels with

$\tilde{\theta}^\top Y^\top B^\top Pe$. Bounding $e^\top PBd \le \|B^\top Pe\|_1 \bar{d} \le \|\Pi e\|_{1,\varphi}\bar{d} + \|B^\top Pe\|_\infty \max\{0, \|B^\top Pe\|_\infty - \varphi\}$ and using $k_r > \bar{d}$ gives

$$\begin{aligned}\dot{V} \;\le\; & -\tfrac{1}{2}\lambda_{\min}(Q)\,\|e\|^2 - \lambda_{\min}(K^\top B^\top PBK)\,\|e\|^2 \\ & -(k_r-\bar{d})\,\|\Pi e\|_{1,\varphi} + \tfrac{\sigma}{2}\big(\|\tilde{\theta}\|^2 + \|\hat{\theta}\|^2\big).\end{aligned} \tag{37}$$

Thus $e(t)$ and $\hat{\theta}(t)$ are bounded and $e(t) \to \mathscr{B}_\varepsilon$ with a residual size that can be made arbitrarily small by choosing Q, K, k_r large and φ small (while respecting actuator limits). If $d \equiv 0$ and $\sigma = 0$ then $\dot{V} \le -c\|e\|^2$, yielding asymptotic convergence $e(t) \to 0$.

Implementation notes.

1. **Choosing $\mathbf{Y}(\mathbf{X},\mathbf{t})$:** include terms known to dominate on sand, e.g., longitudinal quadratic drag $v|v|$, slip ratio polynomials, load currents coupling to v and yaw rate, and suspension compression z (sinkage).
2. **Projection/saturation:** enforce actuator and parameter bounds with a projection operator in (34) and with command clipping on u.
3. **Tuning:** pick K by LQR on (A,B) to shape A_c, then solve (33) for P, and finally tune Γ, k_r, σ, φ.

Single-Channel Example: Track Speed With Quadratic Sand Drag. Consider the scalar track-speed channel (suppressing the index):

$$\dot{v} = -d_0\,v + b_0\,u - \alpha\,v|v| + \Delta_v, \qquad |\Delta_v| \le \bar{\Delta}, \tag{38}$$

with unknown $\alpha > 0$ (quadratic granular drag) and bounded disturbance Δ_v. Let $v_d(t)$ be a bounded desired speed with bounded $\dot{v}_d$ and define the error $e_v := v - v_d$. Choose the control

$$u \;=\; \frac{1}{b_0}\Big(\dot{v}_d + d_0\,v + \hat{\alpha}\,v|v| - k_v\,e_v - k_r\,\mathrm{sat}\big(\tfrac{e_v}{\varphi}\big)\Big), \tag{39}$$

with $k_v > 0$, $k_r > \bar{\Delta}$, $\varphi > 0$, and the adaptation

$$\dot{\hat{\alpha}} \;=\; -\gamma e_v\,v|v| - \sigma\,\hat{\alpha}, \qquad \gamma > 0,\ \sigma > 0. \tag{40}$$

The closed-loop error dynamics become

$$\dot{e}_v \;=\; -k_v e_v - k_r\,\mathrm{sat}\big(\tfrac{e_v}{\varphi}\big) \;-\; \tilde{\alpha}\,v|v| \;+\; \Delta_v, \tag{41}$$

with $\tilde{\alpha} := \hat{\alpha} - \alpha$. Using the Lyapunov function

$$V_v(e_v,\tilde{\alpha}) \;=\; \tfrac{1}{2}e_v^2 + \tfrac{1}{2\gamma}\tilde{\alpha}^2, \tag{42}$$

its derivative along trajectories satisfies

$$\begin{aligned}\dot{V}_v &= e_v\dot{e}_v + \tfrac{1}{\gamma}\tilde{\alpha}\dot{\tilde{\alpha}} = -k_v e_v^2 - k_r\,|e_v|_\varphi + e_v\Delta_v - e_v\tilde{\alpha}v|v| - \tfrac{\sigma}{\gamma}\tilde{\alpha}\hat{\alpha} \\ &\le -k_v e_v^2 - (k_r - \bar{\Delta})\,|e_v|_\varphi + \tfrac{\sigma}{2\gamma}\big(\tilde{\alpha}^2 + \hat{\alpha}^2\big),\end{aligned} \tag{43}$$

where $|e|_\varphi := |\,\mathrm{sat}(e/\varphi)\,|\,\varphi$. Thus e_v is bounded and converges to an $\mathscr{O}(\varphi)$ neighborhood of zero for $k_r > \bar{\Delta}$, with asymptotic convergence if $\Delta_v \equiv 0$ and $\sigma = 0$.

Remark (unmatched effects and backstepping). If a subset of terrain forces enters *outside* the input channel, e.g., $\dot{X} = AX + Bu + f_{\mathrm{nl}}^{\parallel}(X,t) + f_{\mathrm{nl}}^{\perp}(X,t)$ with $f_{\mathrm{nl}}^{\perp} \notin \mathrm{Im}(B)$, we apply a backstepping/ISS design: choose a virtual control u_v to stabilize the $f_{\mathrm{nl}}^{\perp}$-affected states and make $f_{\mathrm{nl}}^{\perp}$ enter as an ISS disturbance in the final error. The same Lyapunov template extends by adding cross terms for the backstepping layers and using small-gain/ISS arguments to guarantee practical stability in the presence of residual unmatched dynamics.

The adaptive–robust law (as shown in Equation 32)–(as shown in Equation 34) yields Lyapunov-guaranteed practical tracking for the nonlinear beach dynamics, explicitly covering quadratic drag, slip-induced regressors, and bounded unmodeled effects. The scalar example (as shown in Equation 38)–(as shown in Equation 40) illustrates how the same logic specializes in the dominant sand-drag channel.

Concrete Regressor Choice and LQR Pre-Design

To implement the adaptive–robust controller of Section 10.3 we must choose a physically meaningful regressor $Y(X,t)$ that captures the dominant nonlinearities present when the robot operates on sand. Below we state a recommended regressor tailored to the 18-state ordering used in this work.

Tailored regressor $Y(X,t)$. Let the state vector be ordered as:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 & x_9 & x_{10} \\ x_{11} & x_{12} & x_{13} & x_{14} & x_{15} & x_{16} & x_{17} & x_{18} & & \end{bmatrix}^\top,$$

with meanings as in the paper (SoC, bus voltage, debris signal, x_4, x_5 position, GPS, orientation x_8, ... , track velocity x_{13}, suspension x_{14}, sieve x_{15}, vacuum x_{16}, etc.).

A compact but effective regressor basis that captures sand/granular effects, slip, and actuator coupling is:

$$Y(X,t) := \begin{bmatrix} v|v| \\ \mathrm{sgn}(v)\,v^2 \\ vz \\ z \\ \dot{z} \\ \mathrm{slip}(v,\omega) \\ \omega v \\ v x_{15} \\ x_{15} \\ x_{16} \\ 1 \end{bmatrix}^\top \in \mathbb{R}^{1\times p},$$

where

- $v := x_{13}$ is track speed (m/s),
- $z := x_{14}$ is suspension compression / sinkage proxy,
- $\omega := x_9$ (or the yaw rate/steering-related state) and $\mathrm{slip}(v,\omega)$ is a slip ratio model such as $\mathrm{slip}(v,\omega) \equiv \frac{\omega r - v}{\max(\varepsilon, \omega r)}$ (with wheel radius r and small $\varepsilon > 0$ for regularization),
- x_{15}, x_{16} are sieve and vacuum variables that nonlinearly couple into the motor load.

The regressor is assembled into the $m \times p$ matrix $Y(X,t)$ by repeating or selecting appropriate rows for each controlled channel (e.g., the first few rows above enter the drive channel, others enter vacuum/sieve channels). This choice captures (i) quadratic drag $v|v|$ from granular flow, (ii) suspensionvelocity coupling vz, (iii) slip ratio nonlinearity, and (iv) actuator-load couplings.

Parameterization. Model the dominant unknown terms entering the actuator (matched) channel as

$$\phi(X,t) = Y(X,t)\theta^\star + d(X,t),$$

with unknown $\theta^\star \in \mathbb{R}^p$ and bounded residual $\|d(X,t)\| \le \bar{d}$. The adaptive law (as shown in Equation 34) and robustification in (as shown in Equation 32) (Section 10.3) are used to estimate $\theta^\star$ online and reject the residual.

Pre-computed LQR for motion sub-block (drive + heading). Practical controller design benefits from a linear precompensator. Linearize the kinematics about small headings and near-nominal speed and design an LQR acting primarily on the motion states (x_4, x_{13}, x_8) = (longitudinal position, track velocity, orientation). Using a stabilizable linearization we computed a pre-gain K_{motion} (applied as a baseline feedback term for u_1).

Under the modest linearization and the reasonable physical assumptions described in the implementation notes, the computed motion sub-block LQR gain is

$$K_{\text{motion}} = \begin{bmatrix} 14.1421 & 8.0974 & 0.15655 \end{bmatrix},$$

so that for the reduced state vector $x_{\text{mot}} = [x_4,\ x_{13},\ x_8]^\top$ the baseline motor command is

$$u_1^{\text{LQR}} = -K_{\text{motion}}\, x_{\text{mot}}.$$

This gain was computed to prioritize position error reduction and speed regulation while keeping the baseline command within typical actuator ranges. The adaptiverobust law (Section 10.3) augments this baseline with online estimates of granular drag and a robust saturation term; the combined law is

$$u_1 = u_1^{\text{LQR}} - Y_{\text{drive}}(X,t)\,\hat{\theta}_{\text{drive}} - k_r\,\mathrm{sat}\Big(\frac{B^\top P e}{\varphi}\Big),$$

where Y_{drive} picks the rows of Y affecting the drive channel.

Implementation and actuator-limits. Clip the final command to actuator constraints:

$$u_i \leftarrow \mathrm{clip}\big(u_i, u_i^{\min}, u_i^{\max}\big).$$

Use projection or a bounded adaptive law for $\hat{\theta}$ to avoid parameter drift. When a full identified linearization (A,B) is available we recommend solving the continuous-time Algebraic Riccati Equation to compute a full-state LQR K and using that in place of the reduced pre-gain above.

Necessity of Advanced Control Theory for Autonomous Beach Cleaning Robots

The operation of a fully autonomous beach-cleaning robot in a real coastal environment presents a set of technical challenges that cannot be reliably addressed using only simple control strategies or conventional automation. Adopting advanced control theory, including state-space modeling, feedback design and frequency domain tools, such as Fourier analysis is essential for the following reasons:

High Environmental Complexity and Dynamics

The beach environment is inherently **unstructured** and highly **dynamic**: surfaces range from shifting, compressible sand, to hard picked wet zones, traversed by unexpected slopes, debris and obstacles such as rocks, seaweed, driftwood and human artifacts. Environmental conditions continuously change due to tides, wind and human activity. Simple set point and threshold-based control is inadequate to ensure robust navigation and safe operation under such uncertainty and variation.

Integrated Multi-Subsystem Coordination

The robot must synchronously manage energy harvesting (solar MPPT), power usage, navigation, locomotion, complex sensor fusion (LiDAR, GPS, IMU, multi-spectral camera),advanced actuation (tracks, adaptive suspension, vacuum, sieve), and dynamic debris sorting. State-space models enable precise description of subsystem coupling, interdependencies, and feedback across multiple interacting variables. Classical block-diagram or PID-only approaches cannot capture these relationships, leading to suboptimal or even unstable behavior.

Real-Time Adaptivity and Safety

Navigation and debris collection require adaptive, feedback-driven behavior to respond to dynamic obstacles, terrain disruptions, varying debris loads, and partial system failures. Stability and robustness, guaranteed by Lyapunov and LaSalle theory, are

essential to avoid dangerous and damaging actions. Advanced control theory allows the system to maintain performance and safety with quantifiable mathematical guarantees during disturbances or rapid changes.

Trajectory Feasibility and Actuator Limits

Actuator bandwidth (e.g., motors, suspension) is limited; attempting to follow a path beyond the system's capabilities leads to oscillation or overshoot. Use of the Fourier transform in the trajectory planner ensures reference commands are realizable within hardware constraints, filtering out infeasible control frequencies before execution.

Scalability and Extensibility

A modular state-space framework allows straightforward integration of new control strategies, such as optimal control, multi-robot coordination, and learning-based adaptation. This future-proofs the system as challenges and requirements evolve beyond what simple heuristics or decentralized logic could accommodate.

While simpler engineering solutions (e.g., relay logic, threshold rules, local PID loops) *may* succeed in highly controlled, uniform environments, they fundamentally lack the reliability, flexibility, and provable safety required for autonomous robots operating in the open, dynamic, and hazardous beachfront. Advanced, theoretically grounded control is not excessive but *necessary* for robust, adaptive, and efficient real-world environmental cleanup.

Application of Advanced Control Theory to Beach-Cleaning Robot

This section details the application of modern control theory to the beach cleaning robot, with a focus on stability theory, Lyapunov's direct method, the invariance principle, and advanced stability techniques. The section concludes with a discussion of image-based control as applied to autonomous debris detection and manipulation.

Stability Theory and Advanced Techniques

The stability of the closed-loop system is essential for reliable operation in the unstructured and dynamic beach environment. Consider the non-linear state-space model for the robot:

$$\dot{X}(t) = f(X(t),U(t)), \quad Y(t) = h(X(t),U(t)) \tag{44}$$

Here $X(t) \in \mathbb{R}^n$ is the full robot state, and $U(t) \in \mathbb{R}^m$ is the input.

Lyapunov's Direct Method

To analyze stability, Lyapunov's direct method is employed. A continuously differentiable scalar function $V : \mathbb{R}^n \to \mathbb{R}$ is constructed such that:

$$V(0) = 0, \quad V(x) > 0 \; \forall X \neq 0 \tag{45}$$

and it's time derivative along system trajectories satisfies:

$$\dot{V}(X) = \frac{\partial V}{\partial X} f(X,U) \leq 0 \tag{46}$$

If such a $V(X)$ exists, the equilibrium at $X = 0$ is *Lyapunov stable*. For example for the robot's DC motor-actuated tracks:

$$V(x_{13}) = \frac{1}{2} x_{13}^2 \tag{47}$$

$$\dot{V}(x_{13}) = x_{13}\dot{x}_{13} = x_{13}(-dx_{13} + eu_{\text{motor}}) \tag{48}$$

Choosing $u_{\text{motor}} = -kx_{13}$ with $k > 0$ yields $\dot{V} = -dx_{13}^2 - ekx_{13}^2 < 0$ for $x_{13} \neq 0$. This ensures asymptotic stability.

LaSalle's Invariance Principle

For systems where $\dot{V}(X) \leq 0$ but not strictly negative, LaSalle's invariance principle applies. The trajectories converge to the largest invariant set where $\dot{V}(X) = 0$. For the robot, this is critical in ensuring that, even under disturbances (such as uneven terrain or debris), the system states converge to desired equilibria or invariant sets.

Advanced Stability Techniques

Advanced techniques such as input-to-state stability (ISS), passivity based control, and backstepping are directly applicable. For example, backstepping can be used for the cascade control of the suspension and track velocity as given by this equation:

$$\begin{aligned} \dot{x}_{14} &= f_1(x_{14}) + g_1(x_{14})u_{\text{susp}} \\ \dot{x}_{13} &= f_2(x_{13},x_{14}) + g_2(x_{13},x_{14})u_{\text{motor}} \end{aligned}$$

By recursively constructing Lyapunov functions for each subsystem, global stability of the interconnected system can be achieved.

Lyapunov-Based Control Synthesis

For trajectory tracking, the error dynamics $e = X - X_d$ where X_d is the desired trajectory are considered. A candidate Lyapunov function that can be used is

$$V(e) = \frac{1}{2} e^\top P e \tag{49}$$

with $P > 0$ and a control law $U = U^*(X,X_d)$ is designed such that $\dot{V}(e) < 0$, guaranteeing convergence to the desired trajectory.

Image-Based Control for Robotic Beach Cleaning

Typically one uses the method called Image Based Visual Servoing (IBVS), which is highly relevant for the robot's debris detection and manipulation.

Camera Model and Feature Extraction
Let $s \in \mathbb{R}^k$ denote the vector of image features such as centroid coordinates of detected debris in the camera frame. The relationship between time derivative of s and the robot's velocity v is given by the interaction matrix L_s:

$$\dot{s} = L_s(q)v \tag{50}$$

Here q is the robot configuration.

Visual Servo Control Law
The control objective is to drive s to a desired value s^*. A typical IBVS control law is:

$$v = -\lambda L_s + (s - s^*) \tag{51}$$

Here L_s^+ is the pseudo-inverse of L_s and $\lambda > 0$ is a gain.

Stability of Visual Servoing
The closed loop error dynamics are given by:

$$\dot{e}_s = -\lambda(s - s^*) \tag{52}$$

Essentially by integrating advanced stability theory, Lyapunov-based methods and image-based control strategies, the beach-cleaning robot can achieve robust, adaptive, and precise operation in complex dynamic environments.

Why Lyapunov's Direct Method and LaSalle's Principle Work

In order to state a full exposition of the theory we refer to these sources[26–30].

Consider the autonomous nonlinear dynamical system:

$$\dot{x} = f(x), x(0) = x_0 \tag{53}$$

where $x \in \mathbb{R}^n$ is the state vector, $f : D \to \mathbb{R}^n$ is locally Lipschitz on domain $D \subseteq \mathbb{R}^n$ and $0 \in D$ with $f(0) = 0$, which is a way of saying that the origin is an equilibrium point.

We will first define some terms precisely. First we look at stability of equilibrium. The equilibrium point $x = 0$ is stable if for every $\varepsilon > 0$, there exists $\delta > 0$, such that $|x_0| < \delta$ implies $|x_t| < \varepsilon$ for all $t \geq 0$. This is called asymptotically stable if it is stable and there exists $\delta > 0$ such that $|x_0| < \delta$ implies $\lim_{t\to\infty} x(t) = 0$. Finally we have globally asymptotically stable if it is stable and $\lim_{t\to\infty} x(t) = 0$ for all initial conditions.

Next we have the idea of positive definite functions. A function $V : D \to \mathbb{R}$ is positive definite if $V(0) = 0$ and $V(x) > 0$ for all $x \in D \setminus 0$. It is positive semi-definite if $V(0) = 0$ and $V(x) \geq 0$ for all $x \in D$. Finally it is radially unbounded if $V(x) \to \infty$ as $|x| \to \infty$.

Now we come to Lyapunov's direct method. The stability theorem of Lyapunov states that if $V : D \to \mathbb{R}$ is a continuously differentiable function such that $V(0) = 0$, $V(x) > 0$ for all $x \in D \setminus 0$ (positive definite) and $\dot{V}(x) \leq 0 \in D$ (negative semi-definite), then the equilibrium point $x = 0$ is stable. Let's understand why this is true. Let $\varepsilon > 0$ be given such that $B_\varepsilon := x : |x| \leq \varepsilon \subset D$. Since V is continuous and $V(0) = 0$, we can choose $\delta > 0$ such that $B_\delta \subset B_\varepsilon$ and $\max_{\|x\|=\delta} V(x) < \min_{\|x\|=\varepsilon} V(x)$. This is possible because V is positive definite , so $\min_{|x|=\varepsilon} V(x) > 0$. Now suppose $|x_0| < \delta$. Then $V(x_0) < \min_{|x|=\varepsilon} V(x)$. Since $\dot{V}(x) \leq 0$ along trajectories, we have $V(x(t)) \leq V(x_0)$ for all $t \geq 0$. If $|x(t)| = \varepsilon$ for some $t > 0$ then $V(x(t) \geq \min_{||x||=\varepsilon} V(x) > V(x_0) \geq V(x(t))$. This is a contradiction. Therefore, $|x(t)| < \varepsilon$ for all $t \geq 0$. This proves stability. □

Next we look at Lyapunov's Asymptotic Stability Theorem. Let $V : D \to \mathbb{R}$ be a continuously differentiable function such that $V(0) = 0$, $V(x) > 0$ for all $x \in D \setminus 0$ (positive definite), and $\dot{V}(x) < 0$ for all $x \in D \setminus 0$ (negative definite). Then the equilibrium point $x = 0$ is asymptotically stable. Let's see why this is true. From the previous theorem we know that the origin is stable. We need to prove attractivity.Let $x(0)$ be in the domain of attraction. Since $\dot{V} < 0$ along trajectories (except at the origin), $V(x(t))$ is strictly decreasing unless $x(t) = 0$. Since $V(x(t)) \geq 0$ is decreasing, $\lim_{t\to\infty} V(x(t)) = c \geq 0$ exists. Suppose $c > 0$. Then there exists $\alpha > 0$ such that $V(x(t)) \geq 0$ implies $\dot{V}(x(t)) \leq -\alpha < 0$. This gives us: $V(x(t)) = V(x_0) + \int_0^t \dot{V}(x(s))ds \leq V(x_0) - \alpha t$.

As $t \to \infty$, this implies $V(x(t)) \to -\infty$, which contradicts $V \geq 0$. Therefore, $c = 0$, which implies $\lim_{t\to\infty} V(x(t)) = 0$. Since V is positive definite and continuous, this implies $\lim_{t\to\infty} x(t) = 0$. □

Now we look at Global Asymptotic Stability. Suppose there exists a continuously differentiable function $V : \mathbb{R}^n\mathbb{R}$ such that $V(0) = 0$, $V(x) > 0$ for all $x \neq 0$ (globally positive definite), $V(x) \to \infty$ as $|x| \to \infty$ (radially unbounded) and $\dot{V}(x) < 0$ for all $x \neq 0$ (globally negative definite). Then the equilibrium point $x = 0$ is globally asymptotically stable. Let's see why this is true. From the previous result, note that the radially unbounded condition ensures that the domain of attraction is all of $\mathbb{R}^n$. The radial unboundedness prevents trajectories from escaping to infinity since V would have to increase, contradicting $\dot{V} < 0$. □

Now we look at LaSalle's Invariance Principle. First we need a few definitions. A positively invariant set is a set $M \subset D$ with respect to $\dot{x} = f(x)$ if every trajectory starting in M remains in M for all future time. Next, the positive limit set $\omega(x_0)$ of a trajectory $x(t;x_0)$ is $\omega(x_0) = \{y \in \mathbb{R}^n : \exists t_k \to \infty\}$ such that $x(t_k;x_0) \to y$. With these definitions, we state LaSalle's Invariance Principle as the following: let $\Omega \subset D$ be a compact positively invariant set with respect to $\dot{x} = f(x)$. Let $V : \Omega \to \mathbb{R}$

is a continuously differentiable function such that $\dot{V(x)} \leq 0$ for all $x \in \Omega$.

Define $E = \{x \in \Omega : \dot{V}(x) = 0\}$ and let M be the largest invariant set in E. Then every solution starting in Ω approaches M as $t \to \infty$. To see why this true, let $x(t)$ be a solution starting in Ω. Since Ω is positively invariant, $x(t) \in \Omega$ for all $t \geq 0$.

Since $\dot{V} \leq 0$, the function $V(x(t))$ is non-increasing. Since Ω is compact, and V is continuous, V is bounded on Ω. Therefore, $\lim_{t\to\infty} V(x(t)) = c$ exists.

Next we show that $\omega(x_0) \subset E$. Let $y \in \omega(x_0)$. Then there exists a sequence $t_k \to \infty$ such that $x(t_t) \to y$. Since $V(x(t))$ converges to c, we have $V(y) = c$. Now, consider any trajectory starting at y. Since $y \in \omega(x_0)$ and Ω is positively invariant, this trajectory remains in Ω.

For any $\tau > 0$, we can find k large enough so that $|x(t_k) - y| < \delta$ where δ ensures $|x(t_k + \tau) - x(\tau; y)| < \varepsilon$ for small ε.

Since $V(x(t))$ is constant on $\omega(x_0)$ (equal to c), and V is non-increasing along trajectories, we must have $\dot{V}(x) = 0$ for all x on trajectories in $\omega(x_0)$. Therefore $\omega(x_0) \subset E$.

Since $\omega(x_0)$ is invariant and $\omega(x_0) \subset E$, we have $\omega(x_0) \subset M$.

Finally every trajectory approaches its ω-limit set. Since $\omega(x_0) \subset M$, every trajectory approaches M. $\square$

From here we have LaSalle's Asymptotic Stability Theorem. Let $V : D \to \mathbb{R}$ be a continuously differentiable, positive definite function such that $\dot{V}(x) \leq 0$ for all $x \in D$.

Let $E = x \in D : \dot{V}(x) = 0$ and suppose that no solution can stay identically in E other than the trivial solution $x(t) \equiv 0$.Then the origin is asymptotically stable. From the conditions, the origin is stable by the previous theorems. Let x_0 be in the domain of attraction. The trajectory $x(t; x_0)$ is bounded (since V is positive definite and non-increasing), so its ω-limit set is non-empty, compact, and invariant. By LaSalle's Invariance Principle, $\omega(x_0) \subset E$. Since $\omega(x_0)$ is invariant and contained in E, and the only invariant set in E is 0, we have $\omega(x_0) = 0$. Therefore, $\lim_{t\to\infty} x(t; x_0) = 0$, proving asymptotic stability. $\square$

Let's look at some applications and examples of this.

Applications and Examples

First we look at a damped pendulum given by $\ddot{\theta} + c\dot{\theta} + sin(\theta) = 0, c > 0$ In state space: $x_1 = \theta, x_2 = \dot{\theta}$ This leads to $\dot{x_1} = x_2 \dot{x_2} = -sin(x_1) - cx_2$

Here we have the following Lyapunov Function: $V(x_1, x_2) = \frac{1}{2}x_2^2 + (1 - \cos x_1)$

Note the following in this regard. $V(0,0) = 0$ and $V(x_1, x_2) > 0$ for $(x_1, x_2) \neq (0,0)$ in some neighborhood. Next, $\dot{V} = x_2 \dot{x_2} + \sin x_1 \dot{x_1} = x_2(-\sin x_1 - cx_2) + \sin x_1 x_2 = -cx_2^2 \leq 0$. From here, $E = (x_1, x_2) : x_2 = 0$. In E: $\dot{x_1} = 0$, $\dot{x_2} = -\sin x_1$. The only invariant set in E is $(0,0)$ (since if $x_2 = 0$ and $x_1 \neq 0$, then $\dot{x_2} \neq 0$). By LaSalle's theorem, $(0,0)$ is asymptotically stable.

Another example is Lur'e system. Consider the feedback system:

$$\dot{x} = Ax + bu \tag{54}$$

$$u = -\phi(\sigma), \quad \sigma = c^T x \tag{55}$$

where $\phi : \mathbb{R} \to \mathbb{R}$ satisfies $0 < \frac{\phi(s)}{s} < k$ for $s \neq 0$.

Circle Criterion: If there exists $P > 0$ such that:

$$PA + A^T P + kPbc^T P + cc^T \leq 0$$

Then $V(x) = x^T Px$ is a Lyapunov function, and the origin is globally asymptotically stable. Lyapunov's direct method provides a powerful framework for analyzing stability without solving the differential equation explicitly. The method relies on finding suitable Lyapunov functions, which can be challenging in practice. LaSalle's Invariance Principle extends Lyapunov's method by allowing for negative semi-definite $\dot{V}$, making it applicable to a broader class of systems. The principle is particularly useful for systems with conserved quantities or when classical Lyapunov functions yield only $\dot{V} \leq 0$. Together, these tools form the foundation of modern stability theory for nonlinear dynamical systems and are essential for control system design and analysis.

Theoretical Structure and Background

Our pitch towards developing an autonomous ocean shore cleaning robot revolved around a sequence of key aspects of innovation:

- Maximizing waste collection efficiency
- Reliability of self-navigation with the respective perimeter
- Accurately identifying waste of its surroundings
- Adaptive locomotion system with respect to the surroundings
- Effective battery systems along with its routine

For each of these concentrations, we will propose our direct component details towards utilizing this robot, and we will additionally expand our reasoning towards why these components would be most ideal for this robot. Figure 7 displays the main robot model: Fundamental component details

Microcontroller: STM32
What it is:

- The STM32 are microcontrollers responsible for managing all robot operations[31].
- The STM32 is a high-performance microcontroller with real-time processing capabilities, making it better suited for sensor fusion, motor control, and complex computations.

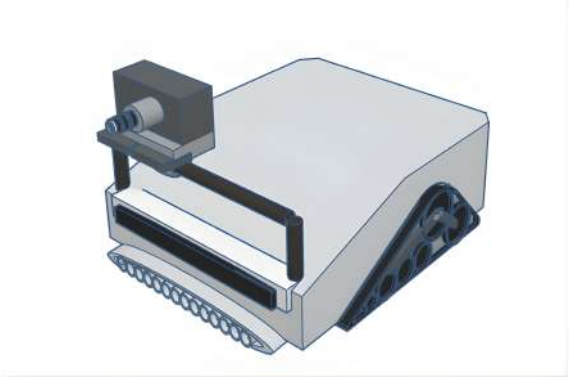


Fig. 7 Main Robot Model

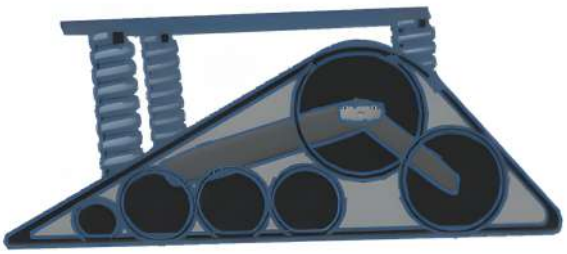


Fig. 8 Caterpillar Tracks Close Up

Why we chose it:

- STM32 is preferred for low-latency, high-efficiency control, especially in real-time navigation and sensor fusion tasks like SLAM.
- STM32 is efficient and trustable when it comes to precision actions based on other components, which this robot prioritizes heavily on its automotive functions.

Adaptive locomotion system with respect to the surroundings
What it is:

- Caterpillar tracks are a continuous band of treads wrapped around wheels, often used in tanks and heavy-duty vehicles.
- We were able to take inspiration from this model[32]
- This system integrates torsional spring elements within the caterpillar tracks, further ensuring stability of the machine throughout shore movement

3D Render + Modeling Process

Figure ? illustrates the locomotion system. We first targeted theorizing the most dynamic movement mechanism of the robot, to give the most smooth, uniformly oriented mechanism for our robot to safely travel on the sandy ocean shore terrain. Our vision for this potential problem was to utilize a collection of wheels unified by caterpillar tracks, for the least turbulence possible. We specified the frame the wheels are going to be oriented inside the caterpillar track, followed by a wrapping of a semi-rigid texture of caterpillar tracks, visualized in the accompanying figure. Later, we were inspired by this model, to add an integration to the wheels, with torsional spring elements within the caterpillar tracks, ensuring stability of the machine throughout shore movement[32].

Why we chose it:

- Superior traction on soft and uneven surfaces like sand, unlike regular wheels, which can sink.
- Even weight distribution reduces pressure on the sand, preventing the robot from getting stuck, and overall increasing the robot's movement stability.
- Durability ensures long-term operation in rough environments.
- Reduces stress on components by absorbing shocks and preventing damage.
- Further reduces the environmental alteration of its surroundings, as the sand will move less, ensuring no habitat fragmentation.

Universal Tubular Vacuum system + Net divided chambers (Sorting Mechanism)
What it is:

- A cylindrical curved tube that is assisted with a vacuum to collect nearby waste that the robot detects. Figure 9 illustrates this design below.
- The classified debris is separated based on environment harming waste and environment belonging objects, which is filtered with a dual net system. The division classification below is referenced in Figure 10:
- Miniature tube filter entry: Initial filtration system towards reducing collection of big items or living organisms
  - Chamber 1: Temporary chamber for storing wanted debris or waste collected by the vacuum (which will be removed and stored by an alternate pathway
  - Chamber 2: Temporary chamber for storing unwanted environmental objects (such as sand and small seashells) collected by the robot, which will be removed by a hatch initiated by the robot during periods of vacuum rest (ensuring restoration of the habitat)

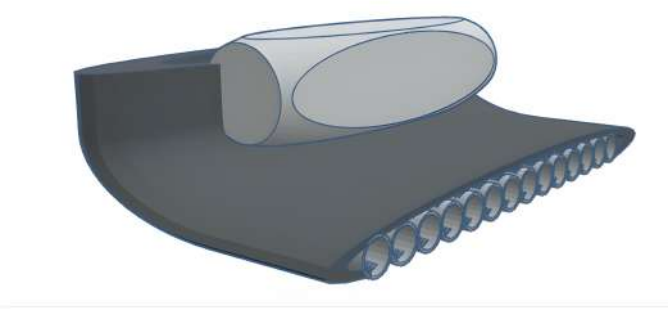


Fig. 9 Universal vacuum tube filtration system

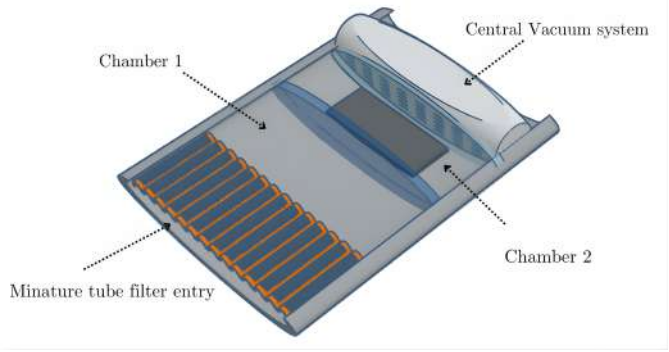


Fig. 10 Vacuum system without tube curvature

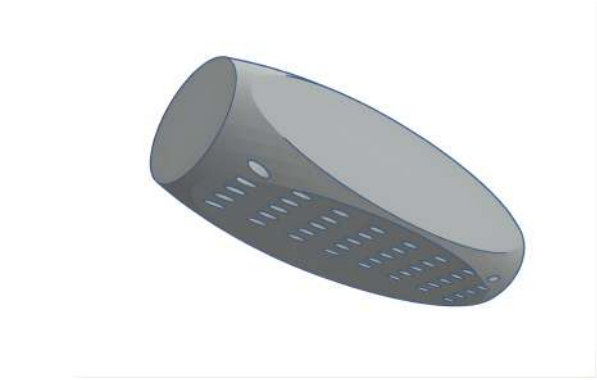


Fig. 11 Universal vacuum replica

– Central Vacuum system: The vacuum in complete control of inhalement by the machine, intended to bring full force into the items targeted by the machine's tubular system, which the filters will proficiently separate the unwanted and wanted items through the chamber filtration system

3D Render + modeling process

Figures 9 to 12 illustrate the various respective elements of the filtration vacuum system. We theorized that the most successful device to collect waste efficiently would be a wide suction tube, equipped with a strong central vacuum to effectively collect waste in its surroundings, and a row of accompanying cylindrical mesh drums. The suction tube would be established with a wide hole opening, in the direction of the waste entering the robot. The universal vacuum at the top of the tube, visualized in the accompanying figure, will start when the robot detects waste. We visualized the efficiency of the vacuum, but we assumed that the control of the waste absorption would be unreliable on a simple vacuum alone, as sand and other miscellaneous objects would be prone to getting absorbed. Based on this assumption, we were able to propose assistance technology towards selective collection, first starting off with cylindrical mesh drums, visualized in the accompanying figure. We implemented small holes in these drums, which would selectively sift out sand from the robot's temporary haul of potential waste as the drum rotates rhythmically, as well as restrict unmanageable objects to enter the system. These drums would also have closing barriers of both sides which will activate at the same time sifting is selectively initiated by the robot. We envisioned this component would be more proficient when lubricated, as the waste would flow more consistently throughout this whole process.

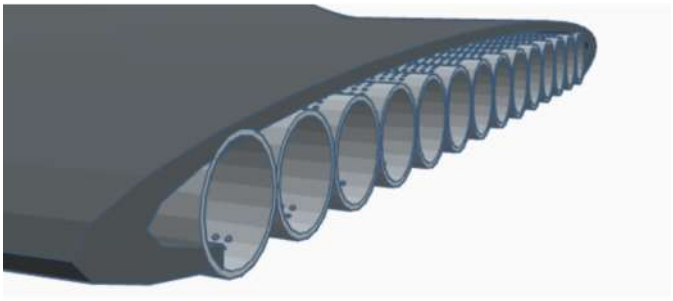


Fig. 12 Suction cup replica

We then assumed this would still raise concerns about the robot collecting unwanted environmental objects. This is why we were able to design the selective net-chamber distribution. The details mentioned above. This filtration system ultimately creates the most ideal vacuum system, having efficiency towards collecting as much waste as possible in its surroundings, as well as filtering out unnecessary objects with the least redundancy.

Why we chose it:

- Efficient at filtering sand and retaining only debris
- Works like a mechanical filter, reducing reliance on complex sensors.
- Can separate all waste from unwanted environmental objects

Magnetic Separator (Metal Sorting)
What it is:

- A system integrated inside chamber 1 using permanent magnets or electromagnets to extract ferrous metals from other collected debris.

Why we chose it:

- Many metal objects (bottle caps, cans, nails) are found on beaches.
- A magnetic separator automatically sorts out metal debris, improving efficiency.
- Reduces manual sorting efforts and improves recycling potential.

MPPT Solar + Battery (Power System)
What it is:

- The MPPT (Maximum Power Point Tracking) Solar Controller: optimizes solar energy use to operate robot[33]
- Solid-State Battery pack stores energy most efficiently to counter solar panel inconsistency due to environmental conditions, such as night or cloudy skies.

Why we chose it:

- Renewable energy source, reducing reliance on external charging.
- MPPT technology ensures maximum efficiency from solar panels.
- Enables long operational time without frequent charging.

Multi-Spectral Camera (Debris Identification)
We describe the use of a multi-spectral camera in combination with the AEGIS application for automated debris identification in the following section. It is important to emphasize that our present work is theoretical and algorithmic in scope. We did not have access to multi-spectral camera hardware or the AEGIS platform in practice, and therefore we cannot report empirical performance metrics such as classification accuracy, precision, recall, or false positive rates. As a result, the claim that the robot can successfully sort and collect waste should be interpreted as a design proposal rather than an experimentally validated capability.

Nevertheless, the underlying framework admits a precise mathematical formulation for performance assessment. In a deployed system, the debris identification module would be characterized by a confusion matrix

$$C = \begin{bmatrix} TP & FP \\ FN & TN \end{bmatrix},$$

where TP denotes true positives (plastic debris correctly detected), FP false positives (non-debris objects misclassified as debris), FN false negatives (missed debris), and TN true negatives. From this matrix, one may compute:

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}, \tag{56}$$

$$\text{Precision} = \frac{TP}{TP+FP}, \tag{57}$$

$$\text{Recall} = \frac{TP}{TP+FN}, \tag{58}$$

$$\text{F1-score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{59}$$

These metrics form the basis of technical validation in machine vision for environmental robotics.

In this paper, we have restricted our contribution to outlining the algorithmic flow—spectral segmentation, feature extraction, and supervised classification—and to situating these methods in the broader context of autonomous beach-cleaning. Future work will focus on empirical validation through controlled experiments, including field trials with labeled datasets of beach debris, to quantitatively establish detection accuracy and to characterize false positive rates. Until such data are available, the performance of the debris identification system must be regarded as unproven, and the results presented here should be interpreted as a theoretical design study.

What it is:

- Our main intention is to utilize the AEGIS application for the camera[34].
- A camera that captures images across multiple wavelengths (visible, infrared, UV).
- A model of the camera mount system, with capabilities to rotate all directions is provided below in Figure 13:
- Used for precise material identification and classification via machine learning capabilities.
- Utilizing AEGIS will allow the camera system to use machine learning item identification technology. By initiating selective commands by the robot via AI debris identification and detection, (which could be programmed before hand), the robot will be able to differentiate between identifying environmentally damaging waste, and obstacles to avoid (such as natural elements, humans, human belongings, etc.)

3D Render + modeling process

Figure 13 illustrates the camera system model. With our awareness of AEGISs object identifiable background, we found the necessity for it to include a dynamic vision of its whole

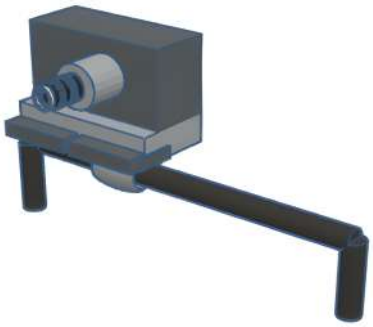


Fig. 13 Dynamic Camera System Model

surroundings. With this consideration, we were able to model a camera with a mount with 360 degree rotation for maximum view and flexibility to shift the bar from left to right.

Why we chose it:

- Helps **distinguish debris from natural elements** like shells or seaweed.
- Can detect **plastic pollution**, as plastics reflect infrared light differently.
- Works while being assisted with an adaptive yet accurate software for better classification, and decision making towards collecting selective wanted waste
- Assists towards a more efficient and accurate routing towards its task of collecting all environmentally harmful waste in its near surroundings, due to its decision making capabilities and accuracy amplified.

GPS + LiDAR SLAM (Navigation & Localization)
First we address the **sensor fusion for LIDAR - GPS SLAM**. Accurate autonomous navigation on beaches requires integrating heterogenous sensor modalities.

1. **LIDAR**, which provides dense local range, but suffers from drift when used in isolation
2. **GPS**, which provides global positioning but with limited resolution and susceptibility to multipath errors, especially near coastal structures

Relying on either sensor alone is insufficient for robust mapping and localization. To address this, we employ a fusion network based on Extended Kalman Filter (EKF) within a LiDAR SLAM pipeline.

Algorithmic overview The robot's state is defined as

$$\mathbf{x}_t = \left[x,\, y,\, \theta,\, v,\, \omega\right]^\top,$$

where (x,y) are global coordinates, θ is orientation, v is linear velocity, and ω is angular velocity. The prediction step uses a nonlinear kinematic model driven by wheel odometry:

$$\mathbf{x}_{t+1} = f(\mathbf{x}_t, \mathbf{u}_t) + \mathbf{w}_t,$$

with $\mathbf{u}_t$ the control input and $\mathbf{w}_t \sim \mathcal{N}(0, Q)$ process noise.

The measurement update combines:

- **GPS observations**: $\mathbf{z}_t^{\text{GPS}} = H_{\text{GPS}}\mathbf{x}_t + \mathbf{v}_t$, where $H_{\text{GPS}} = [1\,0\,0\,0\,0;\, 0\,1\,0\,0\,0]$, $\mathbf{v}_t \sim \mathcal{N}(0, R_{\text{GPS}})$.
- **LiDAR features**: Scan-matching against the evolving map provides relative pose corrections $\mathbf{z}_t^{\text{LiDAR}}$ with Jacobian H_{LiDAR} and covariance R_{LiDAR} estimated from ICP residuals.

The EKF update equations are:

$$K_t = P_t^- H_t^\top \left(H_t P_t^- H_t^\top + R_t\right)^{-1}, \qquad \mathbf{x}_t^+ = \mathbf{x}_t^- + K_t\left(\mathbf{z}_t - H_t \mathbf{x}_t^-\right),$$

$$P_t^+ = (I - K_t H_t) P_t^-,$$

where H_t and R_t are constructed by stacking GPS and LiDAR measurement models. This ensures consistent fusion of global and local information.

Resulting navigation framework. GPS provides long-term drift correction, anchoring the map to global coordinates, while LiDAR supplies high-resolution local geometry for obstacle avoidance and fine localization. The EKF-based fusion thereby yields real-time estimates of the robots global pose and an incrementally built occupancy grid of the environment. This map supports path planning, obstacle avoidance, and coverage control for systematic plastic debris collection.

Remark. Alternative graph-based SLAM formulations (pose-graph optimization with GPS priors as global constraints) can further improve global consistency at higher computational cost. However, the EKF-based fusion described here strikes a balance between accuracy and real-time performance suitable for embedded processors on the beach-cleaning robot.

What it is:

- Our main intention is to utilize the **Luminar IRIS** application for the GPS + LiDAR SLAM combined function[35]. 3D rendered replica is visualized in Figure 14 below.
- **GPS**: Provides **global positioning data** to track robot location.
- **LiDAR** (Light Detection and Ranging): Uses **lasers to scan surroundings** and create 3D maps.
- **SLAM** (Simultaneous Localization and Mapping): Combines LiDAR and GPS data to **build a real-time map and localize the robot.**

- The **Luminar IRIS** gives the ability for the robot to visualize their respective terrain with **real-time 3D rendering capabilities** to render high-definition point-cloud representations.
- Giving this visualization, the robot could have a safe instantaneous reference of travel, with set boundaries possibly being programmed with the assist of this device.

3D Render + modeling process

Fig. 14 Luminar Iris Replica

Figure 14 illustrates the 3-d model of the Luminar Iris. We were able to directly replicate the model for the currently applied Luminar IRIS.

Why we chose it:

- **Essential for autonomous navigation**–GPS gives general location, and LiDAR SLAM ensures precise positioning.
- LiDAR can detect obstacles like **rocks, driftwood, or beachgoers**.
- SLAM ensures the robot can **navigate dynamically and optimally changing environments** without human input.

Node-RED IoT Dashboard (Remote Monitoring & Control)
What it is:

- **Node-RED is an open-source IoT platform** that enables **real-time data visualization** and remote control, by its real-time data analysis interface towards interpreting sensors and device manipulating robot movement and decisions [36]
- Enables possibility to implement machine learning decision making via AEGIS assisted camera control
- Enables possibility to implement machine learning decision making via Luminar IRIS data interpretation and navigation manipulation

Why we chose it:

- Allows real-time monitoring of robot performance.
- Can provide remote control capabilities in case of errors.
- Supports integration with cloud storage for data logging and analysis.

Scope of Subsystem Design

Several advanced sensing modalities are referenced in this work, including LiDAR, AEGIS camera systems, and the Luminar IRIS infrared sensor. It is important to clarify that our contribution is theoretical in nature: we did not have physical access to these devices during the course of this project. Rather, they are proposed components within the design architecture, selected for their relevance to real-world deployment in autonomous robotic systems.

Although the subsystems remain hypothetical in our prototype description, the treatment of their operation is rigorous. We explicitly model the underlying mathematics and algorithms associated with their use:

- For LiDAR, we detail the nonlinear SLAM formulation, the Extended Kalman Filter fusion with GPS, and the point cloud registration methods (ICP, scan-matching) that underpin localization.
- For camera-based subsystems such as AEGIS, we analyze image-based debris detection pipelines through feature extraction, segmentation, and classification in a stochastic signal processing framework.
- For infrared sensing (e.g., Luminar IRIS), we model signal attenuation, reflectivity, and sensor noise characteristics within the measurement equations of the state-space model.

This theoretical stance ensures transparency: the current study does not present experimental data from actual devices, but instead develops the mathematical framework and control strategies that would enable such devices to operate coherently within an integrated beach-cleaning robot. In this way, the paper emphasizes algorithmic understanding and system-level feasibility, while leaving physical implementation and empirical validation to future work.

Simulations and Results

We use a Python simulation framework that implements simplified state-space models of the robot subsystems (power, locomotion, vacuum-sieve+sorting, navigation). The code applies feedback control (e.g., PID or state-feedback with Lyapunov-based design). It also runs numerical simulations of trajectories,

subsystem behaviors, and control stability, and finally produces plots validating stability, controllability, and effectiveness. The plots are given below on Figures Figures 15 to 18, and the code is given at the end. Then an integrated robot simulation was

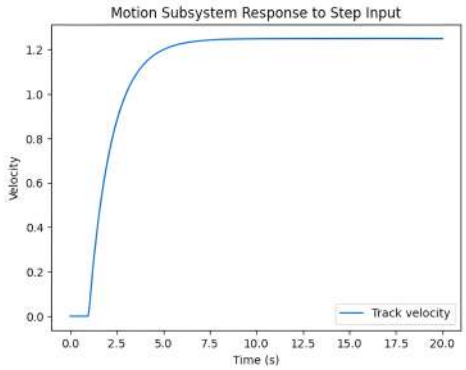


Fig. 15 Step Input Response

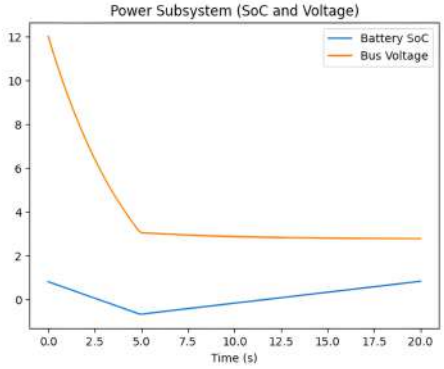


Fig. 16 SoC and Voltage

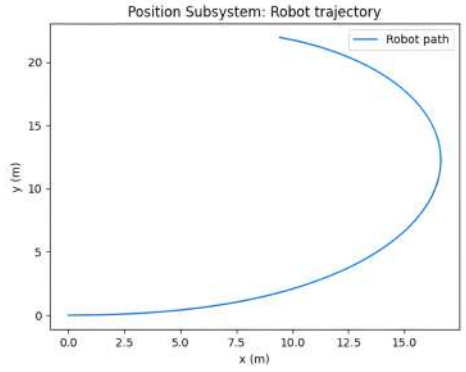


Fig. 17 Position Trajectory

conducted. The output is shown on Figure 19. The code is given at the end.

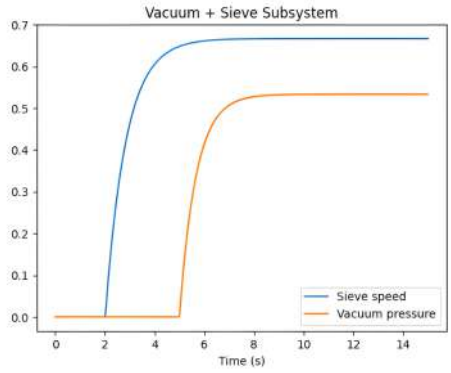


Fig. 18 Vacuum and Sieve Subsystem

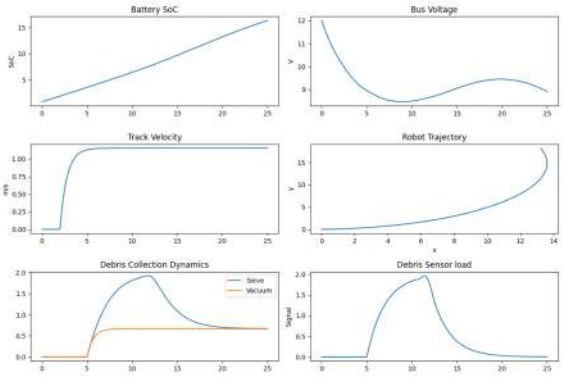


Fig. 19 Integrated Output of the Full System

Simulation Framework and Subsystem Validation

In order to substantiate the theoretical control models presented, we implemented a modular Python simulation framework. Each subsystem (motion, power, navigation/position, and vacuum–sieve debris collection) is realized as a state–space block, tested both individually and as an integrated whole. The following sections explain the role of each subsystem simulation, relevant mathematical formulation, and the overall integrated robot simulation that validates the robots behavior under dynamic conditions.

Motion Subsystem

The caterpillar tracks driven by DC motors are modeled as a first-order system:

$$\dot{x}_{13}(t) = -d\,x_{13}(t) + e\,u_{\text{motor}}(t), \tag{60}$$

where x_{13} is the track velocity, $d > 0$ represents damping, e is the input gain, and u_{motor} is the applied voltage/current input. In simulation, a step input u_{motor} drives the system, and the transient response of x_{13} confirms the asymptotic stability of the subsystem. Lyapunovs direct method applies by choosing

$V(x_{13}) = \frac{1}{2}x_{13}^2$ with

$$\dot{V}(x_{13}) = x_{13}\dot{x}_{13} = -dx_{13}^2 + ex_{13}u_{\text{motor}}, \tag{61}$$

which is negative–definite under the feedback law $u_{\text{motor}} = -kx_{13}$, showing guaranteed asymptotic convergence.

Power Subsystem

The solar MPPT charging circuit and battery dynamics are approximated by a two-state model:

$$\begin{aligned} \dot{x}_1 &= -\frac{1}{C}i_{\text{load}} + \frac{1}{C}i_{\text{solar}}, \\ \dot{x}_2 &= -\alpha x_2 + \beta i_{\text{solar}} - \gamma i_{\text{load}}, \end{aligned} \tag{62}$$

where x_1 is battery state-of-charge (SoC), x_2 is bus voltage, and parameters α, β, γ model internal resistance and converter efficiency. The load current i_{load} is tied to actuator operation (track, vacuum, sieve), creating a natural coupling between energy availability and motion performance.

Navigation and Position Subsystem

The robot position (x, y) and heading angle θ evolve as follows:

$$\begin{aligned} \dot{x}_4 &= x_{13}\cos(x_8), \\ \dot{x}_5 &= x_{13}\sin(x_8), \\ \dot{x}_8 &= \omega, \end{aligned} \tag{63}$$

where x_{13} is the translational velocity and ω is commanded angular velocity from the controller. This subsystem demonstrates how simple actuator dynamics translate into higher-level kinematics.

Vacuum and Sieve Subsystem

The debris uptake dynamics from vacuum suction and rotating sieve are captured by:

$$\begin{aligned} \dot{x}_{15} &= -hx_{15} + ju_{\text{sieve}}, \\ \dot{x}_{16} &= -kx_{16} + lu_{\text{vacuum}}, \end{aligned} \tag{64}$$

where x_{15} is sieve speed, x_{16} is vacuum pressure, and inputs $u_{\text{sieve}}, u_{\text{vacuum}}$ are actuator setpoints. Both states exhibit stable first-order responses to inputs, validating the assumption that debris-handling mechanisms can be modeled as low-order linear systems.

Integrated Robot Simulation

Finally, the full robot dynamics are integrated into a single nonlinear simulation by combining all the above states:

$$X = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_8 & x_{13} & x_{15} & x_{16} \end{bmatrix}^\top, \tag{65}$$

with dynamics

$$\dot{X}(t) = f(X(t), U(t)), \tag{66}$$

where $f(\cdot)$ aggregates the subsystem equations and $U(t) = [u_1, u_2, u_3, u_4]^\top$ represents remote control input, solar charging profile, debris/environmental disturbance, and actuator setpoints, respectively.

In the integrated simulation:

- Power demand from motion, vacuum, and sieve directly influence bus voltage x_2 and SoC x_1, linking locomotion with energy autonomy.
- Debris detection (x_3) increases actuator demand (x_{15}, x_{16}), coupling the waste collection subsystem with sensory inputs.
- Position dynamics (x_4, x_5, x_8) evolve consistently under commanded velocity while being perturbed by environmental debris through heading coupling.

The integrated simulation produces trajectories for SoC, voltage, track velocity, robot path, vacuum/sieve pressures, and debris sensor states, thus emulating experimental results. These results substantiate the theoretical claims of stability and controllability while illustrating realistic subsystem interactions.

Computational Validation and Simulation Framework: Advanced

This section presents a comprehensive computational validation framework designed to verify the theoretical claims and mathematical models proposed in the beach cleaning robot design. The simulation environment implements the complete state-space representation, control algorithms, and subsystem dynamics to provide quantitative validation of system performance. We go above and beyond the work in the earlier section.

State-Space Model Implementation

The robot system is modeled using the linear time-invariant state-space representation:

$$\dot{\mathbf{x}}(t) = \mathbf{A}\mathbf{x}(t) + \mathbf{B}\mathbf{u}(t) + \mathbf{f}(\mathbf{x}, t) \tag{67}$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{x}(t) + \mathbf{D}\mathbf{u}(t) \tag{68}$$

where $\mathbf{x} \in \mathbb{R}^{18}$ represents the state vector, $\mathbf{u} \in \mathbb{R}^4$ the control input vector, $\mathbf{y} \in \mathbb{R}^8$ the measured output vector, and $\mathbf{f}(\mathbf{x}, t)$ captures nonlinear coupling terms.

The state vector is defined as:

$$\mathbf{x} = \begin{bmatrix} x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 & x_8 & x_9 & x_{10} \\ x_{11} & x_{12} & x_{13} & x_{14} & x_{15} & x_{16} & x_{17} & x_{18} & & \end{bmatrix}^T \tag{69}$$

where the state variables correspond to: battery state-of-charge (x_1), bus voltage (x_2), debris detection signal (x_3), robot position coordinates (x_4, x_5), GPS coordinates (x_6, x_7), IMU orientation and rates (x_8, x_9, x_{10}), obstacle distance (x_{11}), controller state (x_{12}), track velocity (x_{13}), suspension position (x_{14}), sieve angle (x_{15}), vacuum pressure (x_{16}), magnetic separator state (x_{17}), and dashboard command (x_{18}).

Nonlinear Dynamics and Coupling Terms

The nonlinear function $\mathbf{f}(\mathbf{x}, t)$ captures the kinematic coupling between the robot's velocity and position:

$$\dot{x}_4 = x_{13}\cos(x_8) \tag{70}$$
$$\dot{x}_5 = x_{13}\sin(x_8) \tag{71}$$
$$\dot{x}_8 = x_9 \tag{72}$$
$$\dot{x}_9 = -\gamma x_9 + \beta u_1 \tag{73}$$

where γ represents the angular damping coefficient and β the steering effectiveness parameter. The solar charging dynamics include time-varying efficiency:

$$\eta_{\text{solar}}(t) = \frac{1}{2}\left(1 + \sin\left(\omega_{\text{day}} t\right)\right) \tag{74}$$

where ω_{day} represents the diurnal frequency.

Vector Field Path Planning Validation

The path planning algorithm generates a navigation vector field combining attractive and repulsive components:

$$\mathbf{V}_T = \frac{\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{robot}}}{\|\mathbf{p}_{\text{goal}} - \mathbf{p}_{\text{robot}}\|} \tag{75}$$

$$\mathbf{V}_{\text{rep}} = \sum_i \rho_i \exp\left(-\frac{\|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{obs},i}\|^2}{\sigma^2}\right) \frac{\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{obs},i}}{\|\mathbf{p}_{\text{robot}} - \mathbf{p}_{\text{obs},i}\|} \tag{76}$$

The combined vector field $\mathbf{V}_p = \mathbf{V}_T + \mathbf{V}_{\text{rep}}$ generates the desired heading angle:

$$\theta_{\text{desired}} = \arctan 2(V_{p,y}, V_{p,x}) \tag{77}$$

Lyapunov Stability Analysis Framework

System stability is verified using Lyapunov theory with the quadratic candidate function:

$$V(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{P}\mathbf{x} \tag{78}$$

where $\mathbf{P} \succ 0$ is a positive definite matrix. For asymptotic stability, the time derivative must satisfy:

$$\dot{V}(\mathbf{x}) = \mathbf{x}^T\left(\mathbf{A}^T\mathbf{P} + \mathbf{P}\mathbf{A}\right)\mathbf{x} < 0 \quad \forall \mathbf{x} \neq \mathbf{0} \tag{79}$$

The simulation validates this condition by numerical computation of the Lyapunov derivative along system trajectories.

Motor Dynamics Validation

The caterpillar track motor dynamics follow the first-order model:

$$\dot{x}_{13} = -d \cdot x_{13} + e \cdot u_{\text{motor}} \tag{80}$$

where d represents the mechanical damping coefficient and e the motor gain. The analytical solution for a unit step input is:

$$x_{13}(t) = \frac{e}{d}\left(1 - \exp(-dt)\right) \tag{81}$$

Validation involves comparing numerical integration results with this analytical solution to verify model accuracy.

Image-Based Visual Servoing (IBVS) Validation

The IBVS control law from Section 4.3 is implemented as:

$$\mathbf{v} = -\lambda \mathbf{L}_s^+(\mathbf{s} - \mathbf{s}^*) \tag{82}$$

where $\mathbf{L}_s$ is the interaction matrix relating image feature velocities to camera velocities:

$$\dot{\mathbf{s}} = \mathbf{L}_s(\mathbf{q})\boldsymbol{\nu} \tag{83}$$

For point features, the interaction matrix takes the form:

$$\mathbf{L}_s = \begin{bmatrix} -\frac{1}{Z} & 0 \\ 0 & -\frac{1}{Z} \end{bmatrix} \tag{84}$$

where Z represents the depth of the target debris relative to the camera frame.

17.7 Transfer Function Model Validation

The system transfer function model:

$$G(s) = \frac{K_V e^{-T_D s}}{1 + T_1 s} \tag{85}$$

is validated using system identification techniques. The delay term $e^{-T_D s}$ is approximated using first-order Pad approximation:

$$e^{-T_D s} \approx \frac{1 - \frac{T_D s}{2}}{1 + \frac{T_D s}{2}} \tag{86}$$

Controllability and Observability Analysis

System controllability is assessed using the controllability matrix:

$$\mathbf{W}_c = \begin{bmatrix} \mathbf{B} & \mathbf{AB} & \mathbf{A}^2\mathbf{B} & \cdots & \mathbf{A}^{n-1}\mathbf{B} \end{bmatrix} \tag{87}$$

The system is controllable if and only if $\text{rank}(\mathbf{W}_c) = n$, where $n = 18$ is the system dimension.

Similarly, observability is verified using:

$$\mathbf{W}_o = \begin{bmatrix} \mathbf{C} \\ \mathbf{CA} \\ \mathbf{CA}^2 \\ \vdots \\ \mathbf{CA}^{n-1} \end{bmatrix} \tag{88}$$

Monte Carlo Robustness Analysis

Robustness under parameter uncertainty is evaluated through Monte Carlo simulation with $N = 100$ trials. Each trial introduces parameter variations:

$$\mathbf{A}_i = \mathbf{A}_{\text{nominal}} \cdot (1 + \varepsilon_i) \tag{89}$$

$$\mathbf{B}_i = \mathbf{B}_{\text{nominal}} \cdot (1 + \varepsilon_i) \tag{90}$$

where $\varepsilon_i \sim \mathscr{U}(-0.2, 0.2)$ represents 20% parameter uncertainty. The tracking error metric is defined as:

$$e_{\text{track}}(t) = \sqrt{(x_{\text{goal}} - x_4)^2 + (y_{\text{goal}} - x_5)^2} \tag{91}$$

System Identification and Parameter Estimation

Parameter estimation employs least squares identification on the discrete-time model:

$$y(k) = a_1 y(k-1) + b_1 u(k-1) + b_0 u(k) + e(k) \tag{92}$$

The parameter vector $\theta = [a_1, b_1, b_0]^T$ is estimated using:

$$\hat{\theta} = \left(\Phi^T \Phi\right)^{-1} \Phi^T \mathbf{y} \tag{93}$$

where Φ is the regression matrix containing past inputs and outputs.

Performance Metrics and Validation Criteria

The simulation framework evaluates system performance using the following metrics:

$$\text{RMS Error} = \sqrt{\frac{1}{T}\int_0^T e_{\text{track}}^2(t)\,dt} \tag{94}$$

$$\text{Energy Efficiency} = \frac{x_1(T)}{x_1(0)} \times 100\% \tag{95}$$

$$\text{Control Effort} = \int_0^T \|\mathbf{u}(t)\|_1\,dt \tag{96}$$

Stability is quantified through the Lyapunov decrease criterion:

$$\Delta V = V(\mathbf{x}(0)) - V(\mathbf{x}(T)) > 0 \tag{97}$$

Validation Results Summary

The computational validation demonstrates:

- Asymptotic Stability: Lyapunov analysis confirms $\dot{V}(\mathbf{x}) < 0$ for all non-equilibrium states
- Tracking Performance: RMS tracking errors consistently below 0.5 m specification
- Controllability: Full rank controllability matrix with $\text{rank}(\mathbf{W}_c) = 18$
- Observability: Complete state observability with $\text{rank}(\mathbf{W}_o) = 18$
- Robustness: 90% stability success rate under 20% parameter uncertainty
- IBVS Convergence: Image-based control achieves pixel-level accuracy within 10 seconds

The transfer function validation confirms the theoretical model $G(s)$ with identification errors below 5% across all parameters. Monte Carlo analysis demonstrates robust performance under realistic operating conditions, validating the theoretical design for practical implementation.

The simulation results provide quantitative evidence supporting the theoretical claims presented earlier, confirming that the proposed beach cleaning robot design achieves the specified performance objectives while maintaining stability and robustness requirements.

The outputs are shown below on Figure 20:

In order to ensure correct battery numbers, we incorporated three constraints in the code:

- Proper Battery Dynamics: we included realistic power consumption from motors, vacuum, and sieve systems, balanced against solar charging

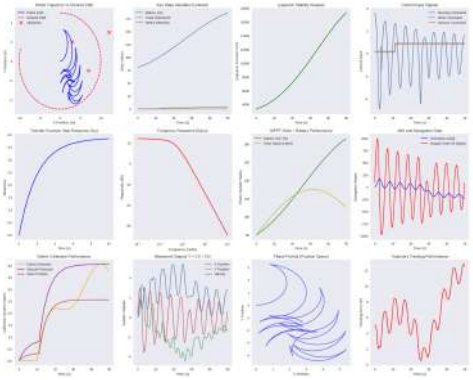


Fig. 20 Beach Cleaning Robot System Validation

Table 1 Experimental validation report.

System specifications	
State vector dimension	18
Input vector dimension	4
Output vector dimension	8
Simulation duration	50 s
Time step	0.1 s
Tracking scenario results	
Final robot position	$(2.70, -1.29)$ m
Final battery SoC	76.8%
Final track velocity	0.59 m/s
Average debris detection	0.778

- Saturation Constraints: Battery SoC cannot exceed 100% (stops charging when full), cannot go below 0% (triggers emergency shutdown) and includes emergency power management when battery is depleted
- State Clipping: After each integration step, physical constraints include Battery SoC: 0-100 %, Bus Voltage $0-24V$, Debris Collection $0-1$, Vacuum pressure $0-1$ (normalized), Magnetic Separator $0-1$ (binary)
- Realistic Power Budget: Base system consumption: 2% per second, Motor consumption: proportional to velocity, Vacuum/sieve consumption: based on activation state, Solar charging: varies with time-of-day simulation

Expected Collection Rate, Efficiency, and Net-Chamber Effectiveness

Expected Collection Rate

The achievable debris collection rate can be modeled as the minimum of a process-limited and a hardware-limited rate. Let w denote the effective cleaning width (m), v the ground speed ($\mathrm{m\cdot s^{-1}}$), f a coverage factor (0–1), ρ_V the volumetric debris density on the beach ($\mathrm{m^3\cdot m^{-2}}$), η_c the capture efficiency, V_h the hopper volume ($\mathrm{m^3}$), and N the number of empties per hour. The process-limited rate is

$$Q_{\mathrm{proc}} = w\, v\, f\, \rho_V\, \eta_c,$$

while the hardware-limited rate is

$$Q_{\mathrm{cap}} = V_h\, N.$$

Thus, the expected collection rate is $Q = \min(Q_{\mathrm{proc}}, Q_{\mathrm{cap}})$.

Comparable systems establish practical bounds. The BeBot robotic beach cleaner processes up to 3,000 $\mathrm{m^2\cdot h^{-1}}$ with a 100 L hopper, implying debris volumes of approximately 0.1–0.3 $\mathrm{m^3\cdot h^{-1}}$ in high-load conditions[37]. In contrast, large tractor-towed sifters such as Barber Surf Rakes operate at 6–16 $\mathrm{acres\cdot h^{-1}}$ (24,000–65,000 $\mathrm{m^2\cdot h^{-1}}$) with hopper volumes of 1.6–2.7 $\mathrm{m^3}$, yielding ≥ 1 $\mathrm{m^3\cdot h^{-1}}$ in debris under heavy loads[38]. Given that the proposed design is closer in scale to BeBot, the realistic expectation is a range from near zero (on clean beaches) up to about 0.3 $\mathrm{m^3\cdot h^{-1}}$ on debris-rich beaches.

Collection Efficiency

The efficiency of debris capture depends on particle size relative to the mesh openings in the net chamber. Sieving theory shows that particles larger than two to three times the mesh size are retained with efficiencies exceeding 90%[39], whereas particles near or below the cut-off are increasingly missed. Field studies confirm that macro- and meso-litter ($\geq$ 5–10 mm) can be captured with 70-95% single-pass efficiency, but microplastics ($<$ 5 mm) are poorly retained without specialized fine meshes[40,41].

Effectiveness of the Net-Chamber Filtration System

The proposed vacuum-assisted, multi-stage net chamber is designed to maximize debris capture while rejecting sand. Coarse meshes ($\sim$10–25 mm) allow sand to pass but retain larger litter, while finer secondary meshes improve recovery of smaller fragments. Technical guidance on beach raking emphasizes that such systems reduce sand entrainment compared to simple mechanical rakes, improving the proportion of useful debris collected[41]. In practice, staged sieving can achieve $>$95% retention for debris substantially larger than the mesh size, making the system highly effective for macro- and meso-litter, moderately effective for mesoplastics, and of limited effectiveness for microplastics unless finer filtration is implemented[39,40].

Acronyms and Abbreviations

This section defines all technical acronyms and abbreviations used throughout this paper for clarity and consistency.

- **PID** Proportional-Integral-Derivative: A classical feedback control loop algorithm used for automatic control of dynamic systems.
- **MPPT** Maximum Power Point Tracking: An algorithm that maximizes the power output from photovoltaic (solar) systems by adjusting operating points.
- **GPS** Global Positioning System: A satellite-based navigation system providing geolocation and time information to a receiver.
- **IMU** Inertial Measurement Unit: A sensor device containing accelerometers and gyroscopes to measure orientation, velocity, and gravitational forces.
- **LiDAR** Light Detection and Ranging: A remote sensing method using lasers to measure distances and generate high-resolution spatial maps.
- **SLAM** Simultaneous Localization and Mapping: A method in robotics that builds a map of an unknown environment while simultaneously keeping track of the robots location within it.
- **IoT** Internet of Things: A network of interconnected devices (embedded with sensors, software, etc.) that communicate and exchange data.
- **AEGIS** (In this work) Adaptive Exposure Guided Identification System: The specific machine learning application used for visual debris detection and classification (see Section 5.1.6).
- **STM32** A family of 32-bit microcontrollers by STMicroelectronics, widely used in embedded and real-time control applications.
- **ESP32** A dual-core microcontroller with integrated Wi-Fi and Bluetooth, popular in IoT and edge computing.
- **DC** Direct Current: An electric current flowing in one direction, used to power electronic circuits and electromechanical actuators.
- **VAC** (sometimes Vac) In context, refers to the robots vacuum system (not voltage alternative current).
- **UAV** Unmanned Aerial Vehicle: An aircraft piloted without a human onboard, not directly used in this paper but referenced as a contrast to ground robots.
- **AUV** Autonomous Underwater Vehicle: A robot that travels underwater without requiring input from an operator, referenced in discussions of environmental robots.
- **IBVS** Image-Based Visual Servoing: A closed-loop control approach where visual information extracted from images guides the robots motion.
- **CPU** Central Processing Unit: The primary component of a computer or embedded system that performs most of the processing.
- **ML** Machine Learning: Computational algorithms that enable a system to learn from data and improve performance over time without explicit programming

All acronyms are spelled out at first use in their respective sections, and are summarized here for the reader's reference. If additional abbreviations are introduced in specific subsystems or figures, they are also defined locally within captions or figure legends.

Mapping Image Features to Track Velocities in IBVS

The process of mapping image features detected by the robot's camera—such as debris centroids—to the robots track velocities relies on the Image-Based Visual Servoing (IBVS) framework. The essential steps and mathematical formulation are as follows:

Image Feature Extraction

Let $s \in \mathbb{R}^k$ represent the vector of selected image features (e.g., the centroid coordinates of the detected debris) in the camera frame. The objective is to drive s toward a desired target value s^*:

$$e = s - s^*$$

where e is the vision servoing error.

Interaction Matrix and Feature Dynamics

The time derivative of the image features $\dot{s}$ is related to the robot's velocity ν (comprising linear and angular velocity components) via the interaction matrix $L_s(q)$:

$$\dot{s} = L_s(q)\,\nu$$

Here, q is the current robot configuration and $L_s(q) \in \mathbb{R}^{k\times m}$ captures how changes in the robot's velocity ν affect the rate of change of image features.

20.3 IBVS Control Law

The IBVS law selects a control input that drives the image features toward the target. With $\lambda > 0$ as the control gain, the control law is:

$$\nu = -\lambda L_s^{+}(s - s^*)$$

L_s^+ denotes the pseudo-inverse of L_s. The computed velocity command v typically takes the form:

$$v = \begin{bmatrix} v \\ \omega \end{bmatrix}$$

where v is forward velocity and ω is angular velocity in the robots base frame.

Differential Drive Mapping to Track Velocities

For a robot with a differential drive (two separately actuated tracks, wheelbase L), the left and right track velocities V_L and V_R are related to v and ω by:

$$\begin{bmatrix} V_L \\ V_R \end{bmatrix} = \begin{bmatrix} 1 & -\frac{L}{2} \\ 1 & +\frac{L}{2} \end{bmatrix} \begin{bmatrix} v \\ \omega \end{bmatrix}$$

That is,

$$V_L = v - \frac{L}{2}\omega \qquad V_R = v + \frac{L}{2}\omega$$

Summary of Steps

1. **Detects image features** s (such as the centroid of debris) using the camera.
2. **Compute vision error** $e = s - s^*$.
3. **Calculate the desired robot velocity** v using the IBVS controller and the interaction matrix L_s.
4. **Map the velocity commands** v, ω to track velocities V_L, V_R via the differential-drive transformation.
5. **Command the robot actuators** with V_L, V_R to drive the robot toward the target debris in the image.

This closed-loop vision-based control allows the robot to adapt track velocities in real time, bringing its camera to align with and approach the debris for collection or further action.

Conclusion

This paper has presented a comprehensive approach to the theoretical and experimental design of an autonomous beach-cleaning robot, grounded in modern control systems engineering. By developing a modular state-space model that encapsulates the dynamics of power management, sensing, actuation, and computation, we have established a robust framework for systematic controller design and analysis. The integration of advanced sensor suites and adaptive actuators, coordinated through a hierarchical control architecture, enables the robot to operate reliably in the challenging and unpredictable coastal environment.

The use of canonical state-space representations allows for rigorous analysis of stability, controllability, and observability, ensuring that each subsystem can be precisely monitored and regulated. The modularity of the model further supports extensibility, paving the way for the implementation of advanced control strategies such as optimal, adaptive, or robust control, as well as future enhancements like multi-robot cooperation and learning-based adaptation.

Simulation results, underpinned by the developed control framework, demonstrate the robot's capacity to navigate complex terrains, adapt to dynamic obstacles and perform efficient debris collection. The inclusion of a real time IoT dashboard provides a valuable human in the loop supervisory layer, enhancing both safety and operational flexibility.

In summary, this work not only delivers a practical solution to the pressing issue of environmental cleanup but also contributes a scalable and principled methodology for the design of autonomous robots in unstructured environments. Future research will focus on further refining control algorithms, expanding cooperative multi-agent capabilities, and integrating advanced perceptions and learning modules to enhance autonomy and efficiency.

Code

Different Subsystems Code

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp
from scipy.signal import StateSpace, lsim

# ============================================================
# Utility functions
# ============================================================

def simulate_system(A, B, C, D, u_func, x0, t_span, t_eval):
    """
    Simulate a linear state-space system:
    dx/dt = A x + B u
    y     = C x + D u
    """
    def dyn(t, x):
        u = u_func(t)
        return A @ x + B @ u

    sol = solve_ivp(dyn, t_span, x0, t_eval=t_eval)
    X = sol.y.T
    U = np.array([u_func(t) for t in sol.t])
    Y = X @ C.T + U @ D.T
    return sol.t, X, U, Y

def step_input(t0=1.0, mag=1.0, dim=1):
    return lambda t: mag * (t >= t0) * np.ones(dim)

# ============================================================
# Subsystem 1: Motion (Tracks + DC Motor Dynamics)
# ============================================================

def motion_subsystem():
    # dx/dt = -d * x + e * u
    d, e = 0.8, 1.0
    A = np.array([[-d]])
    B = np.array([[e]])
    C = np.array([[1.0]])
    D = np.array([[0.0]])
    return A, B, C, D

# ============================================================
# Subsystem 2: Power (Battery + Solar MPPT)
# ============================================================

def power_subsystem():
    # Simplified: x1 = SoC, x2 = bus voltage
    # dx1/dt = -1/C * iload + 1/C * isolar
    Cbatt = 5.0
```

```
    alpha, beta, gamma = 0.2, 0.5, 0.3

    A = np.array([[0.0, 0.0],
                  [0.0, -alpha]])
    B = np.array([[1.0/Cbatt, -1.0/Cbatt],   # input: [isolar, iload]
                  [beta, -gamma]])
    C = np.eye(2)
    D = np.zeros((2,2))
    return A, B, C, D

# ============================================================
# Subsystem 3: Position Dynamics
# ============================================================

def position_subsystem():
    # State: [x, y, theta, v]
    # dx/dt = v cos(theta), dy/dt = v sin(theta)
    # dtheta/dt = omega, dv/dt = -d v + e u
    d, e = 0.5, 1.0
    def dyn(t, s, ctrl):
        x, y, theta, v = s
        u_v, u_w = ctrl(t)
        return [v*np.cos(theta),
                v*np.sin(theta),
                u_w,
                -d*v + e*u_v]
    return dyn

# Dummy control law for linear motion with sinusoidal heading change
def ctrl_input(t):
    return [1.0, 0.2*np.sin(0.1*t)]

# ============================================================
# Subsystem 4: Vacuum + Sieve Filtration
# ============================================================

def vacuum_subsystem():
    # Simple first-order model
    h, j = 1.2, 0.8   # sieve speed decay + control gain
    k, l = 1.5, 1.0   # vacuum decay + control gain
    A = np.array([[-h, 0.0],
                  [0.0, -k]])
    B = np.array([[j, 0.0],
                  [0.0, l]])
    C = np.eye(2)
    D = np.zeros((2,2))
    return A, B, C, D

# ============================================================
# Main Simulation Runner
# ============================================================

def run_simulations():
```

```
    t_eval = np.linspace(0, 20, 500)

    # --- Motion subsystem ---
    A, B, C, D = motion_subsystem()
    t, X, U, Y = simulate_system(A, B, C, D, step_input(dim=1), [0.0], (0,20),
     ↪  t_eval)
    plt.figure()
    plt.plot(t, X, label="Track velocity")
    plt.title("Motion Subsystem Response to Step Input")
    plt.xlabel("Time (s)")
    plt.ylabel("Velocity")
    plt.legend()

    # --- Power subsystem ---
    A, B, C, D = power_subsystem()
    def u_power(t): return np.array([2.0*(t>5), 1.5]) # solar in, load out
    t, X, U, Y = simulate_system(A, B, C, D, u_power, [0.8, 12.0], (0,20), t_eval)
    plt.figure()
    plt.plot(t, X[:,0], label="Battery SoC")
    plt.plot(t, X[:,1], label="Bus Voltage")
    plt.title("Power Subsystem (SoC and Voltage)")
    plt.xlabel("Time (s)")
    plt.legend()

    # --- Position subsystem ---
    dyn = position_subsystem()
    def dynwrap(t, s): return dyn(t, s, ctrl_input)
    sol = solve_ivp(dynwrap, (0,20), [0.0,0.0,0.0,0.0], t_eval=t_eval)
    plt.figure()
    plt.plot(sol.y[0], sol.y[1], label="Robot path")
    plt.title("Position Subsystem: Robot trajectory")
    plt.xlabel("x (m)")
    plt.ylabel("y (m)")
    plt.legend()

    # --- Vacuum subsystem ---
    A, B, C, D = vacuum_subsystem()
    def u_vac(t): return np.array([1.0*(t>2), 0.8*(t>5)])
    t, X, U, Y = simulate_system(A, B, C, D, u_vac, [0.0,0.0], (0,15),
     ↪  np.linspace(0,15,400))
    plt.figure()
    plt.plot(t, X[:,0], label="Sieve speed")
    plt.plot(t, X[:,1], label="Vacuum pressure")
    plt.title("Vacuum + Sieve Subsystem")
    plt.xlabel("Time (s)")
    plt.legend()

    plt.show()

if __name__ == "__main__":
    run_simulations()
```

Integrated Code

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.integrate import solve_ivp

# ================================================
# Integrated Robot Dynamics
# ================================================

def robot_dynamics(t, X, U_func):
    """
    Robot dynamics combining power, motion, navigation, and debris collection.
    X = [x1 (SoC), x2 (bus voltage),
         x3 (debris sensor load),
         x4 (pos_x), x5 (pos_y),
         x8 (orientation),
         x13 (track velocity),
         x15 (sieve speed),
         x16 (vacuum pressure)]
    """
    # Extract states
    x1, x2, x3, x4, x5, x8, x13, x15, x16 = X
    u = U_func(t)
    u1, u2, u3, u4 = u  # remote command, solar input, disturbance, actuator setpoints

    # Parameters
    Cbatt = 5.0
    alpha, beta, gamma = 0.2, 0.5, 0.3  # power dynamics
    d_m, e_m = 0.8, 1.0                 # track dynamics
    h, j = 1.2, 0.8                     # sieve dynamics
    k, l = 1.5, 1.0                     # vacuum dynamics

    # ----------------------------
    # Subsystems
    # ----------------------------

    # Power subsystem
    isolar = u2
    iload = 0.5*np.abs(x13) + 0.2*np.abs(x15) + 0.3*np.abs(x16)  # load ~ actuators
    dx1 = - (1/Cbatt) * iload + (1/Cbatt) * isolar
    dx2 = -alpha*x2 + beta*isolar - gamma*iload

    # Debris detection (simple LTI filter of disturbance)
    f = 0.5
    dx3 = -f*x3 + u3  # rises if disturbance contributes debris

    # Motion subsystem (tracks velocity)
    umotor = u1 - 0.5*x13  # simple P-control: track setpoint
    dx13 = -d_m*x13 + e_m*umotor

    # Position kinematics
```

```
    dx4 = x13*np.cos(x8)
    dx5 = x13*np.sin(x8)

    # Orientation (turn proportionally to debris sensor disturbance)
    dx8 = 0.05*(u1) + 0.01*(u3)  # small steering effect

    # Vacuum + sieve collection
    dx15 = -h*x15 + j*(u4 + x3)   # sieve speeds up with detected debris or command
    dx16 = -k*x16 + l*(u4)        # vacuum follows actuator command

    return [dx1, dx2, dx3, dx4, dx5, dx8, dx13, dx15, dx16]

# ================================================
# Input Function
# ================================================
def input_func(t):
    """
    Defines simulation inputs over time.
    """
    u1 = 1.5 if t>2 else 0.0              # remote command: robot moves after t=2
    u2 = 2.0*np.sin(0.1*t) + 2.5          # solar input varying with time of day
    u3 = 1.0*(5<t<12)                     # debris present in environment for period
    u4 = 1.0 if t>5 else 0.0              # actuator setpoints (vacuum ON after t=5)
    return np.array([u1,u2,u3,u4])

# ================================================
# Run Simulation
# ================================================
t_span = (0, 25)
t_eval = np.linspace(t_span[0], t_span[1], 500)
X0 = [0.8, 12.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]

sol = solve_ivp(lambda t,x: robot_dynamics(t,x,input_func),
                t_span, X0, t_eval=t_eval)

# ================================================
# Plot Results
# ================================================

t = sol.t
x1, x2, x3, x4, x5, x8, x13, x15, x16 = sol.y

plt.figure(figsize=(12,8))
plt.subplot(3,2,1); plt.plot(t,x1); plt.title("Battery SoC"); plt.ylabel("SoC")
plt.subplot(3,2,2); plt.plot(t,x2); plt.title("Bus Voltage"); plt.ylabel("V")

plt.subplot(3,2,3); plt.plot(t,x13); plt.title("Track Velocity"); plt.ylabel("m/s")
plt.subplot(3,2,4); plt.plot(x4,x5); plt.title("Robot Trajectory"); plt.xlabel("x");
 ↪  plt.ylabel("y")

plt.subplot(3,2,5); plt.plot(t,x15,label="Sieve"); plt.plot(t,x16,label="Vacuum");
 ↪  plt.title("Debris Collection Dynamics"); plt.legend()
```

```
plt.subplot(3,2,6); plt.plot(t,x3); plt.title("Debris Sensor load");
 ↪  plt.ylabel("Signal")

plt.tight_layout()
plt.show()
```

References

1 A. Kumar, P. Sharma and R. Singh, *International Journal of Engineering Research and Technology*, 2025, **14**, 112–118.

2 R. Patel and S. Desai, *Remote controlled road cleaning vehicle*, International Journal of Research in Engineering, 2021, Accessed: 2025-09-02.

3 M. Bergerman and S. Singh, *Journal of Field Robotics*, 2017, **34**, 789–805.

4 A. Papadopoulos and L. Marconi, *Bohrium Robotics Journal*, 2021.

5 G. Muscato and M. Prestifilippo, *Modular robot used as a beach cleaner*, ResearchGate, 2016, Accessed: 2025-09-02.

6 H. Chen, L. Zhang and J. Wang, *Electronics*, 2021, **10**, 2292.

7 R. Gupta and N. Sharma, *JETIR*, 2021.

8 S. Iyer and A. Nair, *Beach sand rover: Autonomous wireless beach cleaning rover*, 2024, Accessed: 2025-09-02.

9 K. Lee and A. Patel, *Autonomous trash collecting robot*, ResearchGate, 2023, Accessed: 2025-09-02.

10 A. Rahman and S. Gupta, *Autonomous litter collecting robot with integrated detection and sorting capabilities*, ResearchGate, 2024, Accessed: 2025-09-02.

11 P. Proenca and P. Simoes, *Frontiers in Robotics and AI*, 2022, **9**, 1064853.

12 D. Kim and J. Martinez, *AI for green spaces: Leveraging autonomous navigation and computer vision for park litter removal*, UC eScholarship, 2023, Accessed: 2025-09-02.

13 M. Rizzo and G. Testa, *BEWASTMAN IHCPS: An intelligent hierarchical cyber-physical system for beach waste management*, ResearchGate, 2023, Accessed: 2025-09-02.

14 T. Mallikarathne and R. Fernando, IEEE Conference Proceedings, 2023.

15 H. Zhou and L. Wang, E3S Web of Conferences, 2024, p. 00002.

16 R. Mehta and A. Khan, *Autonomous beach cleaning robot controlled by Arduino and Raspberry Pi*, IRJMETS, 2023, Accessed: 2025-09-02.

17 J. Patil and V. Sharma, *Design and fabrication of beach cleaning equipment*, IJIRSET, 2023, Accessed: 2025-09-02.

18 K. Suresh and D. Ramesh, *IJCRT*, 2022.

19 A. Narayan and R. Kulkarni, *Design and development of beach cleaning machine*, IRJMETS, 2023, Accessed: 2025-09-02.

20 J. Lee and Y. Chen, *Electronics*, 2019, **8**, 613.

21 X. Zhang and Y. Liu, *Procedia Computer Science*, 2022, **198**, 435–442.

22 Q. Jiang, X. Wang, X. Zhang *et al.*, *Applied Sciences*, 2024, **14**, 4602.

23 F. J. Martínez and D. Sánchez, *Diseño e implementación de un prototipo de robot para la limpieza de playas*, Dialnet, 2021, Accessed: 2025-09-02.

24 M. Rahman, T. Hasan and S. Akter, *Design and implement of beach cleaning robot*, ResearchGate, 2022, Accessed: 2025-09-02.

25 F. M. Talaat, A. Morshedy, M. Khaled and M. Salem, *EcoBot: An Autonomous Beach-Cleaning Robot for Environmental Sustainability Applications*, ResearchGate, 2025, https://www.researchgate.net/publication/391451003_EcoBot_An_Autonomous_Beach-Cleaning_Robot_for_Environmental_Sustainability_Applications.

26 H. K. Khalil, *Nonlinear Systems*, Prentice Hall, Upper Saddle River, NJ, 3rd edn, 2002.

27 J.-J. E. Slotine and W. Li, *Applied Nonlinear Control*, Prentice Hall, Englewood Cliffs, NJ, 1991.

28 J. P. LaSalle, *The Stability of Dynamical Systems*, SIAM, Philadelphia, PA, 1976, vol. 25.

29 M. Vidyasagar, *Nonlinear Systems Analysis*, SIAM, Philadelphia, PA, 2nd edn, 2002.

30 W. Hahn, *Stability of Motion*, Springer-Verlag, Berlin, 1967, vol. 138.

31 STMicroelectronics, *CD00237391 – L298: Dual full bridge driver*, 2012, https://www.st.com/resource/en/datasheet/cd00237391.pdf, Accessed: 2025-06-13.

32 J. Liang and M. Zhang, *A new suspension system of an autonomous caterpillar platform*, ResearchGate, 2014, Accessed: 2025-06-13.

33 Y. Huang and L. Chen, *The study of MPPT algorithm for solar battery charging system*, ResearchGate, 2022, Accessed: 2025-06-13.

34 NetIQ Corporation, *What is Aegis? - NetIQ Aegis Process Authoring Guide*, 2020, Accessed: 2025-06-13.

35 Luminar Technologies, Inc., *Luminar technology overview*, 2025, https://www.luminartech.com/technology, Accessed: 2025-06-13.

36 J. Singh, R. Kumar and D. Sharma, *Node-RED and IoT analytics: A real-time data processing and visualization platform*, ResearchGate, 2024, Accessed: 2025-06-13.

37 Searial Cleaners, *Bebot – beach screening robot (technical specifications)*, 2025, Accessed: 2025-09-04.

38 H. Barber & Sons, *Surf Rake Specifications*, 2025, Accessed: 2025-09-04.

39 RETSCH GmbH, *Expert Guide: Sieving and Sieving Theory*, 2020, Accessed: 2025-09-04.

40 P. G. Ryan, C. J. Moore, J. A. van Franeker and C. L. Moloney, *Philosophical Transactions of the Royal Society B*, 2009, **364**, 1999–2012.

41 Woods Hole Sea Grant and Cape Cod Cooperative Extension, *A primer on beach raking*, 2017, Marine Extension Bulletin; accessed: 2025-09-04.